

# IPL\_DataAnalysis\_Case\_Study

June 7, 2026

## 1 IPL Data Scientist Hiring

Position: Senior Data Scientist - IPL Analytics Team

Dataset: Indian IPL Dataset 2008-2024

Submission: Jupyter Notebook with code, visualizations, and insights

### 1.1 Background

The Indian Premier League (IPL) is seeking a talented Data Scientist to join our analytics team. Your role will involve analyzing match data, player performance, fan sentiment, and providing strategic insights to team management.

### 1.2 Dataset Information

**Key Columns:** - **id**: A unique identifier assigned to each IPL match. - **season**: The year in which the IPL season took place. - **city**: The city where the match was played. - **date**: The date on which the match was held. - **match\_type**: Type of the match (e.g., League, Playoff, Final). - **player\_of\_match**: The player who was awarded “Player of the Match” for their performance. - **venue**: The stadium or ground where the match was played. - **team1**: The first team listed in the fixture. - **team2**: The second team listed in the fixture. - **toss\_winner**: The team that won the toss before the match began. - **toss\_decision**: The decision made by the toss-winning team—either to bat or to field first. - **winner**: The team that won the match. - **result**: The method of victory—either by runs, wickets, or other (e.g., tie, no result). - **result\_margin**: The margin by which the match was won (e.g., number of runs or wickets). - **target\_runs**: The number of runs set as a target for the chasing team. - **target\_overs**: The number of overs available to chase the target. - **super\_over**: Indicates whether the match was decided by a super over (‘Y’ for Yes, ‘N’ for No). - **method**: The method used to decide the match result if it was interrupted (e.g., Duckworth–Lewis method). - **umpire1**: Name of the first on-field umpire officiating the match. - **umpire2**: Name of the second on-field umpire officiating the match. —

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

## 2 FEATURES mentioned in questions are high level example. You can create more features to make your model robust

### 2.1 Question 1: Data Preprocessing & Feature Engineering

#### 2.1.1 Task:

1. Load the IPL dataset and perform comprehensive EDA
2. Handle missing values appropriately with justification
3. Create these new features:(These are examples you can create more along with this)
  - `home_advantage`: Boolean indicating if team1 is playing in their home city(optional can be done with assumption)
  - `venue_matches_team1_prior / venue_matches_team2_prior`: number of matches the team has played at this same venue before the current match date
  - `match_importance`: Categorical (league/playoff/final) based on date and season
  - `toss_advantage`: Whether toss winner won the match
  - `season_phase`: Early/Mid/Late season

#### 2.1.2 Deliverables:

- Clean dataset with no missing values
- Visualization showing distribution of matches across venues
- Statistical summary of win percentages for toss winners

*# Your code here*

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('/content/drive/MyDrive/IPL_2008-2024.csv')
print(df.shape)
df.head()
```

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1095 entries, 0 to 1094
Data columns (total 20 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               1095 non-null   int64
1   season          1095 non-null   int64
2   city            1044 non-null   object
3   date            1095 non-null   object
4   match_type      1095 non-null   object
```

```

5  player_of_match  1090 non-null  object
6  venue           1095 non-null  object
7  team1           1095 non-null  object
8  team2           1095 non-null  object
9  toss_winner     1095 non-null  object
10 toss_decision   1095 non-null  object
11 winner          1090 non-null  object
12 result          1095 non-null  object
13 result_margin   1076 non-null  float64
14 target_runs     1092 non-null  float64
15 target_overs    1092 non-null  float64
16 super_over      1095 non-null  object
17 method          21 non-null   object
18 umpire1         1095 non-null  object
19 umpire2         1095 non-null  object
dtypes: float64(3), int64(2), object(15)
memory usage: 171.2+ KB

```

```
[4]: df.describe(include='all')
```

```

[4]:
count      1.095000e+03  1095.000000  1044      1095      1095
unique           NaN           NaN      36      823        8
top           NaN           NaN  Mumbai  2008-04-26  League
freq           NaN           NaN      173        2     1029
mean      9.048283e+05  2016.126027      NaN      NaN      NaN
std       3.677402e+05    4.946940      NaN      NaN      NaN
min       3.359820e+05  2008.000000      NaN      NaN      NaN
25%       5.483315e+05  2012.000000      NaN      NaN      NaN
50%       9.809610e+05  2016.000000      NaN      NaN      NaN
75%      1.254062e+06  2021.000000      NaN      NaN      NaN
max      1.426312e+06  2024.000000      NaN      NaN      NaN

```

```

count      player_of_match      venue      team1 \
unique           291           58           19
top      AB de Villiers  Eden Gardens  Royal Challengers Bangalore
freq           25           77           135
mean           NaN           NaN           NaN
std           NaN           NaN           NaN
min           NaN           NaN           NaN
25%           NaN           NaN           NaN
50%           NaN           NaN           NaN
75%           NaN           NaN           NaN
max           NaN           NaN           NaN

```

```

team2      toss_winner toss_decision      winner      result \

```

|        |                |                |       |                |         |
|--------|----------------|----------------|-------|----------------|---------|
| count  | 1095           | 1095           | 1095  | 1090           | 1095    |
| unique | 19             | 19             | 2     | 19             | 4       |
| top    | Mumbai Indians | Mumbai Indians | field | Mumbai Indians | wickets |
| freq   | 138            | 143            | 704   | 144            | 578     |
| mean   | NaN            | NaN            | NaN   | NaN            | NaN     |
| std    | NaN            | NaN            | NaN   | NaN            | NaN     |
| min    | NaN            | NaN            | NaN   | NaN            | NaN     |
| 25%    | NaN            | NaN            | NaN   | NaN            | NaN     |
| 50%    | NaN            | NaN            | NaN   | NaN            | NaN     |
| 75%    | NaN            | NaN            | NaN   | NaN            | NaN     |
| max    | NaN            | NaN            | NaN   | NaN            | NaN     |

|        |               |             |              |            |          |
|--------|---------------|-------------|--------------|------------|----------|
|        | result_margin | target_runs | target_overs | super_over | method \ |
| count  | 1076.000000   | 1092.000000 | 1092.000000  | 1095       | 21       |
| unique | NaN           | NaN         | NaN          | 2          | 1        |
| top    | NaN           | NaN         | NaN          | N          | D/L      |
| freq   | NaN           | NaN         | NaN          | 1081       | 21       |
| mean   | 17.259294     | 165.684066  | 19.759341    | NaN        | NaN      |
| std    | 21.787444     | 33.427048   | 1.581108     | NaN        | NaN      |
| min    | 1.000000      | 43.000000   | 5.000000     | NaN        | NaN      |
| 25%    | 6.000000      | 146.000000  | 20.000000    | NaN        | NaN      |
| 50%    | 8.000000      | 166.000000  | 20.000000    | NaN        | NaN      |
| 75%    | 20.000000     | 187.000000  | 20.000000    | NaN        | NaN      |
| max    | 146.000000    | 288.000000  | 20.000000    | NaN        | NaN      |

|        |              |         |
|--------|--------------|---------|
|        | umpire1      | umpire2 |
| count  | 1095         | 1095    |
| unique | 62           | 62      |
| top    | AK Chaudhary | S Ravi  |
| freq   | 115          | 83      |
| mean   | NaN          | NaN     |
| std    | NaN          | NaN     |
| min    | NaN          | NaN     |
| 25%    | NaN          | NaN     |
| 50%    | NaN          | NaN     |
| 75%    | NaN          | NaN     |
| max    | NaN          | NaN     |

```
[5]: df.isnull().sum().sort_values(ascending=False)
```

```
[5]: method          1074
city                51
result_margin       19
player_of_match      5
winner              5
target_runs          3
target_overs         3
```

```

id                0
date              0
season            0
venue             0
match_type        0
toss_decision      0
toss_winner        0
team2             0
team1             0
result            0
super_over         0
umpire1           0
umpire2           0
dtype: int64

```

```

[6]: ##check duplicates
duplicates = df.duplicated().sum()
print("Duplicate Rows:", duplicates)
##drop duplicates if they exist
df = df.drop_duplicates()

```

Duplicate Rows: 0

```

[7]: ##Handle missing values
missing = pd.DataFrame({
    'Missing Count': df.isnull().sum(),
    'Missing %': round(df.isnull().mean()*100,2)
})

missing[missing['Missing Count']>0]

```

```

[7]:
      Missing Count  Missing %
city              51         4.66
player_of_match     5         0.46
winner             5         0.46
result_margin      19         1.74
target_runs         3         0.27
target_overs        3         0.27
method            1074        98.08

```

```

[8]: df['city'] = df['city'].fillna('Unknown')
df['winner'] = df['winner'].fillna('No Result')
df['player_of_match'] = df['player_of_match'].fillna('No Award')

```

```

[9]: ## Team name standardization
team_mapping = {
    'Delhi Daredevils': 'Delhi Capitals',

```

```

    'Kings XI Punjab':'Punjab Kings'
}

cols = [
    'team1',
    'team2',
    'winner',
    'toss_winner'
]

for col in cols:
    df[col] = df[col].replace(team_mapping)

```

```

[10]: ##Exploratory Data Analysis
season_matches = df.groupby('season').size()

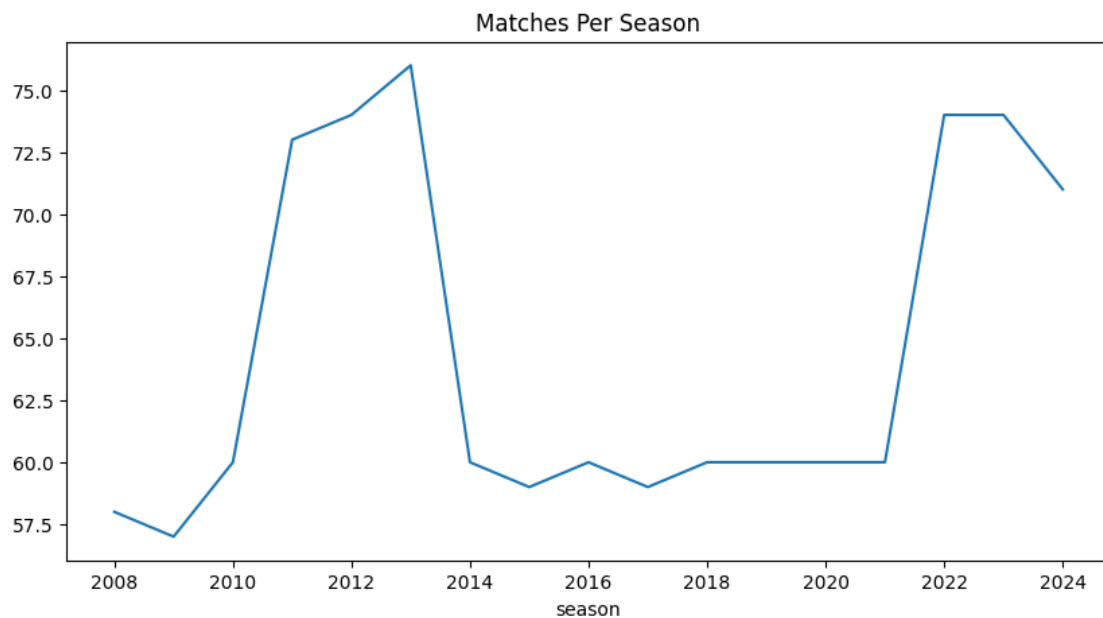
season_matches.plot(
    figsize=(10,5),
    title='Matches Per Season'
)

```

```

[10]: <Axes: title={'center': 'Matches Per Season'}, xlabel='season'>

```



```

[11]: team_wins = df['winner'].value_counts()

team_wins.head(10)

```

```

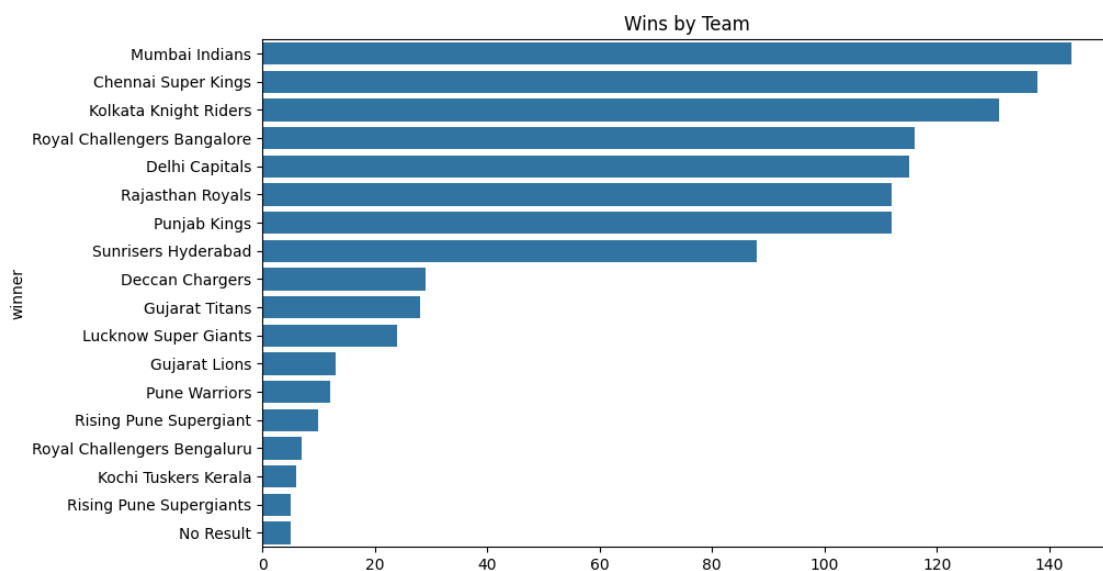
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

sns.barplot(
    x=team_wins.values,
    y=team_wins.index
)

plt.title('Wins by Team')
plt.show()

```



```

[12]: ## Toss Impact
df['toss_match_win'] = (
    df['toss_winner']==df['winner']
).astype(int)
df['toss_match_win'].mean()*100

```

```

[12]: np.float64(50.593607305936075)

```

```

[13]: ## Feature Engineering
## home_advantage

home_city_map = {
    'Mumbai Indians': 'Mumbai',
    'Chennai Super Kings': 'Chennai',
    'Royal Challengers Bengaluru': 'Bangalore',

```

```

'Kolkata Knight Riders': 'Kolkata',
'Rajasthan Royals': 'Jaipur',
'Punjab Kings': 'Chandigarh',
'Delhi Capitals': 'Delhi',
'Sunrisers Hyderabad': 'Hyderabad',
'Lucknow Super Giants': 'Lucknow',
'Gujarat Titans': 'Ahmedabad'
}

```

```

[14]: df['home_advantage_team1'] = (
        df['city'] == df['team1'].map(home_city_map)
    ).astype(int)

df['home_advantage_team2'] = (
        df['city'] == df['team2'].map(home_city_map)
    ).astype(int)

df.head(10)

```

```

[14]:
   id  season  city  date  match_type  player_of_match \
0  335982   2008  Bangalore  2008-04-18  League  BB McCullum
1  335983   2008  Chandigarh  2008-04-19  League  MEK Hussey
2  335984   2008    Delhi  2008-04-19  League  MF Maharooof
3  335985   2008   Mumbai  2008-04-20  League  MV Boucher
4  335986   2008  Kolkata  2008-04-20  League  DJ Hussey
5  335987   2008   Jaipur  2008-04-21  League  SR Watson
6  335988   2008  Hyderabad  2008-04-22  League  V Sehwag
7  335989   2008   Chennai  2008-04-23  League  ML Hayden
8  335990   2008  Hyderabad  2008-04-24  League  YK Pathan
9  335991   2008  Chandigarh  2008-04-25  League  KC Sangakkara

   venue  team1 \
0  M Chinnaswamy Stadium  Royal Challengers Bangalore
1  Punjab Cricket Association Stadium, Mohali  Punjab Kings
2  Feroz Shah Kotla  Delhi Capitals
3  Wankhede Stadium  Mumbai Indians
4  Eden Gardens  Kolkata Knight Riders
5  Sawai Mansingh Stadium  Rajasthan Royals
6  Rajiv Gandhi International Stadium, Uppal  Deccan Chargers
7  MA Chidambaram Stadium, Chepauk  Chennai Super Kings
8  Rajiv Gandhi International Stadium, Uppal  Deccan Chargers
9  Punjab Cricket Association Stadium, Mohali  Punjab Kings

   team2  toss_winner ... \
0  Kolkata Knight Riders  Royal Challengers Bangalore ...
1  Chennai Super Kings  Chennai Super Kings ...
2  Rajasthan Royals  Rajasthan Royals ...

```



|   |                             |                  |     |
|---|-----------------------------|------------------|-----|
| 3 | Royal Challengers Bangalore | Mumbai Indians   | ... |
| 4 | Deccan Chargers             | Deccan Chargers  | ... |
| 5 | Punjab Kings                | Punjab Kings     | ... |
| 6 | Delhi Capitals              | Deccan Chargers  | ... |
| 7 | Mumbai Indians              | Mumbai Indians   | ... |
| 8 | Rajasthan Royals            | Rajasthan Royals | ... |
| 9 | Mumbai Indians              | Mumbai Indians   | ... |

|   | result_margin | target_runs | target_overs | super_over | method | umpire1   | \ |
|---|---------------|-------------|--------------|------------|--------|-----------|---|
| 0 | 140.0         | 223.0       | 20.0         | N          | NaN    | Asad Rauf |   |
| 1 | 33.0          | 241.0       | 20.0         | N          | NaN    | MR Benson |   |
| 2 | 9.0           | 130.0       | 20.0         | N          | NaN    | Aleem Dar |   |
| 3 | 5.0           | 166.0       | 20.0         | N          | NaN    | SJ Davis  |   |
| 4 | 5.0           | 111.0       | 20.0         | N          | NaN    | BF Bowden |   |
| 5 | 6.0           | 167.0       | 20.0         | N          | NaN    | Aleem Dar |   |
| 6 | 9.0           | 143.0       | 20.0         | N          | NaN    | IL Howell |   |
| 7 | 6.0           | 209.0       | 20.0         | N          | NaN    | DJ Harper |   |
| 8 | 3.0           | 215.0       | 20.0         | N          | NaN    | Asad Rauf |   |
| 9 | 66.0          | 183.0       | 20.0         | N          | NaN    | Aleem Dar |   |

|   | umpire2         | toss_match_win | home_advantage_team1 | home_advantage_team2 |
|---|-----------------|----------------|----------------------|----------------------|
| 0 | RE Koertzen     | 0              | 0                    | 0                    |
| 1 | SL Shastri      | 1              | 1                    | 0                    |
| 2 | GA Pratap Kumar | 0              | 1                    | 0                    |
| 3 | DJ Harper       | 0              | 1                    | 0                    |
| 4 | K Hariharan     | 0              | 1                    | 0                    |
| 5 | RB Tiffin       | 0              | 1                    | 0                    |
| 6 | AM Saheba       | 0              | 0                    | 0                    |
| 7 | GA Pratap Kumar | 0              | 1                    | 0                    |
| 8 | MR Benson       | 1              | 0                    | 0                    |
| 9 | AM Saheba       | 0              | 1                    | 0                    |

[10 rows x 23 columns]

```
[15]: ## venue_matches_team1_prior
matches_long = pd.concat([
    df[['date', 'venue', 'team1']]
        .rename(columns={'team1': 'team'}),
    df[['date', 'venue', 'team2']]
        .rename(columns={'team2': 'team'})
])

matches_long = matches_long.sort_values('date')
matches_long
```

```
[15]:      date      venue \
0  2008-04-18  M Chinnaswamy Stadium
```

|      |            |                                            |
|------|------------|--------------------------------------------|
| 0    | 2008-04-18 | M Chinnaswamy Stadium                      |
| 2    | 2008-04-19 | Feroz Shah Kotla                           |
| 1    | 2008-04-19 | Punjab Cricket Association Stadium, Mohali |
| 2    | 2008-04-19 | Feroz Shah Kotla                           |
| ...  | ...        | ...                                        |
| 1092 | 2024-05-22 | Narendra Modi Stadium, Ahmedabad           |
| 1093 | 2024-05-24 | MA Chidambaram Stadium, Chepauk, Chennai   |
| 1093 | 2024-05-24 | MA Chidambaram Stadium, Chepauk, Chennai   |
| 1094 | 2024-05-26 | MA Chidambaram Stadium, Chepauk, Chennai   |
| 1094 | 2024-05-26 | MA Chidambaram Stadium, Chepauk, Chennai   |

|      | team                        |
|------|-----------------------------|
| 0    | Royal Challengers Bangalore |
| 0    | Kolkata Knight Riders       |
| 2    | Rajasthan Royals            |
| 1    | Chennai Super Kings         |
| 2    | Delhi Capitals              |
| ...  | ...                         |
| 1092 | Royal Challengers Bengaluru |
| 1093 | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         |
| 1094 | Sunrisers Hyderabad         |
| 1094 | Kolkata Knight Riders       |

[2190 rows x 3 columns]

```
[16]: ## Prior matches at the venue

df = df.sort_values('date')

venue_team_count = {}

team1_prior = []
team2_prior = []

for _, row in df.iterrows():

    key1 = (row['team1'], row['venue'])
    key2 = (row['team2'], row['venue'])

    team1_prior.append(
        venue_team_count.get(key1, 0)
    )

    team2_prior.append(
        venue_team_count.get(key2, 0)
    )
```

```

venue_team_count[key1] = (
    venue_team_count.get(key1, 0) + 1
)

venue_team_count[key2] = (
    venue_team_count.get(key2, 0) + 1
)

df['venue_matches_team1_prior'] = team1_prior
df['venue_matches_team2_prior'] = team2_prior

df

```

```

[16]:
      id  season    city    date  match_type  player_of_match \
0    335982   2008  Bangalore  2008-04-18    League    BB McCullum
1    335983   2008  Chandigarh  2008-04-19    League    MEK Hussey
2    335984   2008    Delhi  2008-04-19    League    MF Maharroof
3    335985   2008    Mumbai  2008-04-20    League    MV Boucher
4    335986   2008   Kolkata  2008-04-20    League    DJ Hussey
...    ...    ...    ...    ...    ...    ...
1090  1426307   2024  Hyderabad  2024-05-19    League  Abhishek Sharma
1091  1426309   2024  Ahmedabad  2024-05-21  Qualifier 1    MA Starc
1092  1426310   2024  Ahmedabad  2024-05-22  Eliminator    R Ashwin
1093  1426311   2024    Chennai  2024-05-24  Qualifier 2  Shahbaz Ahmed
1094  1426312   2024    Chennai  2024-05-26    Final    MA Starc

      venue \
0          M Chinnaswamy Stadium
1  Punjab Cricket Association Stadium, Mohali
2          Feroz Shah Kotla
3          Wankhede Stadium
4          Eden Gardens
...    ...
1090  Rajiv Gandhi International Stadium, Uppal, Hyd...
1091          Narendra Modi Stadium, Ahmedabad
1092          Narendra Modi Stadium, Ahmedabad
1093          MA Chidambaram Stadium, Chepauk, Chennai
1094          MA Chidambaram Stadium, Chepauk, Chennai

      team1      team2 \
0  Royal Challengers Bangalore  Kolkata Knight Riders
1          Punjab Kings  Chennai Super Kings
2          Delhi Capitals  Rajasthan Royals
3          Mumbai Indians  Royal Challengers Bangalore
4  Kolkata Knight Riders  Deccan Chargers
...    ...    ...

```

|      |                             |                       |
|------|-----------------------------|-----------------------|
| 1090 | Punjab Kings                | Sunrisers Hyderabad   |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals      |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals      |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders |

|      | toss_winner                 | ... | target_overs | super_over | method | \ |
|------|-----------------------------|-----|--------------|------------|--------|---|
| 0    | Royal Challengers Bangalore | ... | 20.0         | N          | NaN    |   |
| 1    | Chennai Super Kings         | ... | 20.0         | N          | NaN    |   |
| 2    | Rajasthan Royals            | ... | 20.0         | N          | NaN    |   |
| 3    | Mumbai Indians              | ... | 20.0         | N          | NaN    |   |
| 4    | Deccan Chargers             | ... | 20.0         | N          | NaN    |   |
| ...  | ...                         | ... | ...          | ...        | ...    |   |
| 1090 | Punjab Kings                | ... | 20.0         | N          | NaN    |   |
| 1091 | Sunrisers Hyderabad         | ... | 20.0         | N          | NaN    |   |
| 1092 | Rajasthan Royals            | ... | 20.0         | N          | NaN    |   |
| 1093 | Rajasthan Royals            | ... | 20.0         | N          | NaN    |   |
| 1094 | Sunrisers Hyderabad         | ... | 20.0         | N          | NaN    |   |

|      | umpire1               | umpire2              | toss_match_win | \ |
|------|-----------------------|----------------------|----------------|---|
| 0    | Asad Rauf             | RE Koertzen          | 0              |   |
| 1    | MR Benson             | SL Shastri           | 1              |   |
| 2    | Aleem Dar             | GA Pratapkumar       | 0              |   |
| 3    | SJ Davis              | DJ Harper            | 0              |   |
| 4    | BF Bowden             | K Hariharan          | 0              |   |
| ...  | ...                   | ...                  | ...            |   |
| 1090 | Nitin Menon           | VK Sharma            | 0              |   |
| 1091 | AK Chaudhary          | R Pandit             | 0              |   |
| 1092 | KN Ananthapadmanabhan | MV Saidharshan Kumar | 1              |   |
| 1093 | Nitin Menon           | VK Sharma            | 0              |   |
| 1094 | J Madanagopal         | Nitin Menon          | 0              |   |

|      | home_advantage_team1 | home_advantage_team2 | venue_matches_team1_prior | \ |
|------|----------------------|----------------------|---------------------------|---|
| 0    | 0                    | 0                    | 0                         |   |
| 1    | 1                    | 0                    | 0                         |   |
| 2    | 1                    | 0                    | 0                         |   |
| 3    | 1                    | 0                    | 0                         |   |
| 4    | 1                    | 0                    | 0                         |   |
| ...  | ...                  | ...                  | ...                       |   |
| 1090 | 0                    | 1                    | 1                         |   |
| 1091 | 0                    | 0                    | 2                         |   |
| 1092 | 0                    | 0                    | 1                         |   |
| 1093 | 0                    | 0                    | 7                         |   |
| 1094 | 0                    | 0                    | 8                         |   |

| venue_matches_team2_prior |
|---------------------------|
| 0                         |

```

1          0
2          0
3          0
4          0
...
1090      12
1091      3
1092      3
1093      2
1094      5

```

[1095 rows x 25 columns]

```

[17]: def classify_match_importance(x):

    if x == 'League':
        return 'League'

    elif 'Final' in str(x):
        return 'Final'

    else:
        return 'Playoff'

df['match_importance'] = (
    df['match_type']
    .apply(classify_match_importance)
)
df

```

```

[17]:      id  season   city   date  match_type  player_of_match \
0    335982   2008  Bangalore  2008-04-18    League    BB McCullum
1    335983   2008  Chandigarh  2008-04-19    League    MEK Hussey
2    335984   2008    Delhi  2008-04-19    League    MF Maharooof
3    335985   2008   Mumbai  2008-04-20    League    MV Boucher
4    335986   2008  Kolkata  2008-04-20    League    DJ Hussey
...
1090  1426307   2024  Hyderabad  2024-05-19    League  Abhishek Sharma
1091  1426309   2024  Ahmedabad  2024-05-21  Qualifier 1        MA Starc
1092  1426310   2024  Ahmedabad  2024-05-22  Eliminator        R Ashwin
1093  1426311   2024   Chennai  2024-05-24  Qualifier 2  Shahbaz Ahmed
1094  1426312   2024   Chennai  2024-05-26    Final        MA Starc

                                venue \
0                                M Chinnaswamy Stadium
1    Punjab Cricket Association Stadium, Mohali
2                                Feroz Shah Kotla

```

|      |                                                   |
|------|---------------------------------------------------|
| 3    | Wankhede Stadium                                  |
| 4    | Eden Gardens                                      |
| ...  | ...                                               |
| 1090 | Rajiv Gandhi International Stadium, Uppal, Hyd... |
| 1091 | Narendra Modi Stadium, Ahmedabad                  |
| 1092 | Narendra Modi Stadium, Ahmedabad                  |
| 1093 | MA Chidambaram Stadium, Chepauk, Chennai          |
| 1094 | MA Chidambaram Stadium, Chepauk, Chennai          |

|      | team1                       | team2 \                     |
|------|-----------------------------|-----------------------------|
| 0    | Royal Challengers Bangalore | Kolkata Knight Riders       |
| 1    | Punjab Kings                | Chennai Super Kings         |
| 2    | Delhi Capitals              | Rajasthan Royals            |
| 3    | Mumbai Indians              | Royal Challengers Bangalore |
| 4    | Kolkata Knight Riders       | Deccan Chargers             |
| ...  | ...                         | ...                         |
| 1090 | Punjab Kings                | Sunrisers Hyderabad         |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders       |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals            |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders       |

|      | toss_winner                 | ... | super_over | method \ |
|------|-----------------------------|-----|------------|----------|
| 0    | Royal Challengers Bangalore | ... | N          | NaN      |
| 1    | Chennai Super Kings         | ... | N          | NaN      |
| 2    | Rajasthan Royals            | ... | N          | NaN      |
| 3    | Mumbai Indians              | ... | N          | NaN      |
| 4    | Deccan Chargers             | ... | N          | NaN      |
| ...  | ...                         | ... | ...        | ...      |
| 1090 | Punjab Kings                | ... | N          | NaN      |
| 1091 | Sunrisers Hyderabad         | ... | N          | NaN      |
| 1092 | Rajasthan Royals            | ... | N          | NaN      |
| 1093 | Rajasthan Royals            | ... | N          | NaN      |
| 1094 | Sunrisers Hyderabad         | ... | N          | NaN      |

|      | umpire1               | umpire2              | toss_match_win \ |
|------|-----------------------|----------------------|------------------|
| 0    | Asad Rauf             | RE Koertzen          | 0                |
| 1    | MR Benson             | SL Shastri           | 1                |
| 2    | Aleem Dar             | GA Pratapkumar       | 0                |
| 3    | SJ Davis              | DJ Harper            | 0                |
| 4    | BF Bowden             | K Hariharan          | 0                |
| ...  | ...                   | ...                  | ...              |
| 1090 | Nitin Menon           | VK Sharma            | 0                |
| 1091 | AK Chaudhary          | R Pandit             | 0                |
| 1092 | KN Ananthapadmanabhan | MV Saidharshan Kumar | 1                |
| 1093 | Nitin Menon           | VK Sharma            | 0                |
| 1094 | J Madanagopal         | Nitin Menon          | 0                |

|      | home_advantage_team1 | home_advantage_team2 | venue_matches_team1_prior | \ |
|------|----------------------|----------------------|---------------------------|---|
| 0    | 0                    | 0                    | 0                         |   |
| 1    | 1                    | 0                    | 0                         |   |
| 2    | 1                    | 0                    | 0                         |   |
| 3    | 1                    | 0                    | 0                         |   |
| 4    | 1                    | 0                    | 0                         |   |
| ...  | ...                  | ...                  | ...                       |   |
| 1090 | 0                    | 1                    | 1                         |   |
| 1091 | 0                    | 0                    | 2                         |   |
| 1092 | 0                    | 0                    | 1                         |   |
| 1093 | 0                    | 0                    | 7                         |   |
| 1094 | 0                    | 0                    | 8                         |   |

|      | venue_matches_team2_prior | match_importance |
|------|---------------------------|------------------|
| 0    | 0                         | League           |
| 1    | 0                         | League           |
| 2    | 0                         | League           |
| 3    | 0                         | League           |
| 4    | 0                         | League           |
| ...  | ...                       | ...              |
| 1090 | 12                        | League           |
| 1091 | 3                         | Playoff          |
| 1092 | 3                         | Playoff          |
| 1093 | 2                         | Playoff          |
| 1094 | 5                         | Final            |

[1095 rows x 26 columns]

```
[18]: ##toss advantage
df['toss_advantage'] = (
    df['toss_winner'] == df['winner']
).astype(int)

df.head(10)
```

|   | id     | season | city       | date       | match_type | player_of_match | \ |
|---|--------|--------|------------|------------|------------|-----------------|---|
| 0 | 335982 | 2008   | Bangalore  | 2008-04-18 | League     | BB McCullum     |   |
| 1 | 335983 | 2008   | Chandigarh | 2008-04-19 | League     | MEK Hussey      |   |
| 2 | 335984 | 2008   | Delhi      | 2008-04-19 | League     | MF Maharoof     |   |
| 3 | 335985 | 2008   | Mumbai     | 2008-04-20 | League     | MV Boucher      |   |
| 4 | 335986 | 2008   | Kolkata    | 2008-04-20 | League     | DJ Hussey       |   |
| 5 | 335987 | 2008   | Jaipur     | 2008-04-21 | League     | SR Watson       |   |
| 6 | 335988 | 2008   | Hyderabad  | 2008-04-22 | League     | V Sehwag        |   |
| 7 | 335989 | 2008   | Chennai    | 2008-04-23 | League     | ML Hayden       |   |
| 8 | 335990 | 2008   | Hyderabad  | 2008-04-24 | League     | YK Pathan       |   |
| 9 | 335991 | 2008   | Chandigarh | 2008-04-25 | League     | KC Sangakkara   |   |

|   | venue                                      | team1 \                     |
|---|--------------------------------------------|-----------------------------|
| 0 | M Chinnaswamy Stadium                      | Royal Challengers Bangalore |
| 1 | Punjab Cricket Association Stadium, Mohali | Punjab Kings                |
| 2 | Feroz Shah Kotla                           | Delhi Capitals              |
| 3 | Wankhede Stadium                           | Mumbai Indians              |
| 4 | Eden Gardens                               | Kolkata Knight Riders       |
| 5 | Sawai Mansingh Stadium                     | Rajasthan Royals            |
| 6 | Rajiv Gandhi International Stadium, Uppal  | Deccan Chargers             |
| 7 | MA Chidambaram Stadium, Chepauk            | Chennai Super Kings         |
| 8 | Rajiv Gandhi International Stadium, Uppal  | Deccan Chargers             |
| 9 | Punjab Cricket Association Stadium, Mohali | Punjab Kings                |

|   | team2                       | toss_winner                 | ... method \ |
|---|-----------------------------|-----------------------------|--------------|
| 0 | Kolkata Knight Riders       | Royal Challengers Bangalore | ... NaN      |
| 1 | Chennai Super Kings         | Chennai Super Kings         | ... NaN      |
| 2 | Rajasthan Royals            | Rajasthan Royals            | ... NaN      |
| 3 | Royal Challengers Bangalore | Mumbai Indians              | ... NaN      |
| 4 | Deccan Chargers             | Deccan Chargers             | ... NaN      |
| 5 | Punjab Kings                | Punjab Kings                | ... NaN      |
| 6 | Delhi Capitals              | Deccan Chargers             | ... NaN      |
| 7 | Mumbai Indians              | Mumbai Indians              | ... NaN      |
| 8 | Rajasthan Royals            | Rajasthan Royals            | ... NaN      |
| 9 | Mumbai Indians              | Mumbai Indians              | ... NaN      |

|   | umpire1   | umpire2        | toss_match_win | home_advantage_team1 \ |
|---|-----------|----------------|----------------|------------------------|
| 0 | Asad Rauf | RE Koertzen    | 0              | 0                      |
| 1 | MR Benson | SL Shastri     | 1              | 1                      |
| 2 | Aleem Dar | GA Pratapkumar | 0              | 1                      |
| 3 | SJ Davis  | DJ Harper      | 0              | 1                      |
| 4 | BF Bowden | K Hariharan    | 0              | 1                      |
| 5 | Aleem Dar | RB Tiffin      | 0              | 1                      |
| 6 | IL Howell | AM Saheba      | 0              | 0                      |
| 7 | DJ Harper | GA Pratapkumar | 0              | 1                      |
| 8 | Asad Rauf | MR Benson      | 1              | 0                      |
| 9 | Aleem Dar | AM Saheba      | 0              | 1                      |

|   | home_advantage_team2 | venue_matches_team1_prior | venue_matches_team2_prior \ |
|---|----------------------|---------------------------|-----------------------------|
| 0 | 0                    | 0                         | 0                           |
| 1 | 0                    | 0                         | 0                           |
| 2 | 0                    | 0                         | 0                           |
| 3 | 0                    | 0                         | 0                           |
| 4 | 0                    | 0                         | 0                           |
| 5 | 0                    | 0                         | 0                           |
| 6 | 0                    | 0                         | 0                           |
| 7 | 0                    | 0                         | 0                           |
| 8 | 0                    | 1                         | 0                           |



9

0

1

0

|   | match_importance | toss_advantage |
|---|------------------|----------------|
| 0 | League           | 0              |
| 1 | League           | 1              |
| 2 | League           | 0              |
| 3 | League           | 0              |
| 4 | League           | 0              |
| 5 | League           | 0              |
| 6 | League           | 0              |
| 7 | League           | 0              |
| 8 | League           | 1              |
| 9 | League           | 0              |

[10 rows x 27 columns]

```
[19]: ##season_phase
      ## Match number within season
      df = df.sort_values(['season', 'date'])

      df['season_match_number'] = (
          df.groupby('season')
            .cumcount() + 1
      )
      ## Total matches in season

      df['season_total_matches'] = (
          df.groupby('season')['id']
            .transform('count')
      )

      ## match progress

      df['season_progress'] = (
          df['season_match_number']
          / df['season_total_matches']
      )

      def season_phase(x):

          if x <= 0.33:
              return 'Early'

          elif x <= 0.66:
              return 'Mid'

          else:
```

```

        return 'Late'

df['season_phase'] = (
    df['season_progress']
    .apply(season_phase)
)

df

```

```

[19]:
      id  season    city    date  match_type  player_of_match \
0    335982   2008  Bangalore  2008-04-18    League    BB McCullum
1    335983   2008  Chandigarh  2008-04-19    League    MEK Hussey
2    335984   2008    Delhi  2008-04-19    League    MF Maharooof
3    335985   2008   Mumbai  2008-04-20    League    MV Boucher
4    335986   2008   Kolkata  2008-04-20    League    DJ Hussey
...    ...    ...    ...    ...    ...    ...
1090  1426307   2024  Hyderabad  2024-05-19    League  Abhishek Sharma
1091  1426309   2024  Ahmedabad  2024-05-21  Qualifier 1    MA Starc
1092  1426310   2024  Ahmedabad  2024-05-22  Eliminator    R Ashwin
1093  1426311   2024   Chennai  2024-05-24  Qualifier 2  Shahbaz Ahmed
1094  1426312   2024   Chennai  2024-05-26    Final    MA Starc

```

```

                                venue \
0                                M Chinnaswamy Stadium
1    Punjab Cricket Association Stadium, Mohali
2                                Feroz Shah Kotla
3                                Wankhede Stadium
4                                Eden Gardens
...                                ...
1090  Rajiv Gandhi International Stadium, Uppal, Hyd...
1091                                Narendra Modi Stadium, Ahmedabad
1092                                Narendra Modi Stadium, Ahmedabad
1093    MA Chidambaram Stadium, Chepauk, Chennai
1094    MA Chidambaram Stadium, Chepauk, Chennai

```

```

                                team1                                team2 \
0    Royal Challengers Bangalore    Kolkata Knight Riders
1                                Punjab Kings    Chennai Super Kings
2                                Delhi Capitals    Rajasthan Royals
3                                Mumbai Indians  Royal Challengers Bangalore
4    Kolkata Knight Riders    Deccan Chargers
...    ...    ...
1090                                Punjab Kings    Sunrisers Hyderabad
1091    Sunrisers Hyderabad    Kolkata Knight Riders
1092  Royal Challengers Bengaluru    Rajasthan Royals
1093    Sunrisers Hyderabad    Rajasthan Royals
1094    Sunrisers Hyderabad    Kolkata Knight Riders

```

|      | toss_winner                 | ... | home_advantage_team1 | \ |
|------|-----------------------------|-----|----------------------|---|
| 0    | Royal Challengers Bangalore | ... | 0                    |   |
| 1    | Chennai Super Kings         | ... | 1                    |   |
| 2    | Rajasthan Royals            | ... | 1                    |   |
| 3    | Mumbai Indians              | ... | 1                    |   |
| 4    | Deccan Chargers             | ... | 1                    |   |
| ...  | ...                         | ... | ...                  |   |
| 1090 | Punjab Kings                | ... | 0                    |   |
| 1091 | Sunrisers Hyderabad         | ... | 0                    |   |
| 1092 | Rajasthan Royals            | ... | 0                    |   |
| 1093 | Rajasthan Royals            | ... | 0                    |   |
| 1094 | Sunrisers Hyderabad         | ... | 0                    |   |

|      | home_advantage_team2 | venue_matches_team1_prior | \ |
|------|----------------------|---------------------------|---|
| 0    | 0                    | 0                         |   |
| 1    | 0                    | 0                         |   |
| 2    | 0                    | 0                         |   |
| 3    | 0                    | 0                         |   |
| 4    | 0                    | 0                         |   |
| ...  | ...                  | ...                       |   |
| 1090 | 1                    | 1                         |   |
| 1091 | 0                    | 2                         |   |
| 1092 | 0                    | 1                         |   |
| 1093 | 0                    | 7                         |   |
| 1094 | 0                    | 8                         |   |

|      | venue_matches_team2_prior | match_importance | toss_advantage | \ |
|------|---------------------------|------------------|----------------|---|
| 0    | 0                         | League           | 0              |   |
| 1    | 0                         | League           | 1              |   |
| 2    | 0                         | League           | 0              |   |
| 3    | 0                         | League           | 0              |   |
| 4    | 0                         | League           | 0              |   |
| ...  | ...                       | ...              | ...            |   |
| 1090 | 12                        | League           | 0              |   |
| 1091 | 3                         | Playoff          | 0              |   |
| 1092 | 3                         | Playoff          | 1              |   |
| 1093 | 2                         | Playoff          | 0              |   |
| 1094 | 5                         | Final            | 0              |   |

|     | season_match_number | season_total_matches | season_progress | season_phase |
|-----|---------------------|----------------------|-----------------|--------------|
| 0   | 1                   | 58                   | 0.017241        | Early        |
| 1   | 2                   | 58                   | 0.034483        | Early        |
| 2   | 3                   | 58                   | 0.051724        | Early        |
| 3   | 4                   | 58                   | 0.068966        | Early        |
| 4   | 5                   | 58                   | 0.086207        | Early        |
| ... | ...                 | ...                  | ...             | ...          |

|      |    |    |          |      |
|------|----|----|----------|------|
| 1090 | 67 | 71 | 0.943662 | Late |
| 1091 | 68 | 71 | 0.957746 | Late |
| 1092 | 69 | 71 | 0.971831 | Late |
| 1093 | 70 | 71 | 0.985915 | Late |
| 1094 | 71 | 71 | 1.000000 | Late |

[1095 rows x 31 columns]

```
[20]: ## Recent form
from collections import defaultdict

team_history = defaultdict(list)

team1_recent_form = []
team2_recent_form = []

for _, row in df.iterrows():

    t1 = row['team1']
    t2 = row['team2']
    winner = row['winner']

    t1_form = (
        np.mean(team_history[t1][-5:])
        if len(team_history[t1]) > 0
        else 0.5
    )

    t2_form = (
        np.mean(team_history[t2][-5:])
        if len(team_history[t2]) > 0
        else 0.5
    )

    team1_recent_form.append(t1_form)
    team2_recent_form.append(t2_form)

    team_history[t1].append(
        1 if winner == t1 else 0
    )

    team_history[t2].append(
        1 if winner == t2 else 0
    )

df['team1_recent_form'] = team1_recent_form
df['team2_recent_form'] = team2_recent_form
```

df

```
[20]:
```

|      | id      | season | city       | date       | match_type  | player_of_match \ |
|------|---------|--------|------------|------------|-------------|-------------------|
| 0    | 335982  | 2008   | Bangalore  | 2008-04-18 | League      | BB McCullum       |
| 1    | 335983  | 2008   | Chandigarh | 2008-04-19 | League      | MEK Hussey        |
| 2    | 335984  | 2008   | Delhi      | 2008-04-19 | League      | MF Maharooof      |
| 3    | 335985  | 2008   | Mumbai     | 2008-04-20 | League      | MV Boucher        |
| 4    | 335986  | 2008   | Kolkata    | 2008-04-20 | League      | DJ Hussey         |
| ...  | ...     | ...    | ...        | ...        | ...         | ...               |
| 1090 | 1426307 | 2024   | Hyderabad  | 2024-05-19 | League      | Abhishek Sharma   |
| 1091 | 1426309 | 2024   | Ahmedabad  | 2024-05-21 | Qualifier 1 | MA Starc          |
| 1092 | 1426310 | 2024   | Ahmedabad  | 2024-05-22 | Eliminator  | R Ashwin          |
| 1093 | 1426311 | 2024   | Chennai    | 2024-05-24 | Qualifier 2 | Shahbaz Ahmed     |
| 1094 | 1426312 | 2024   | Chennai    | 2024-05-26 | Final       | MA Starc          |

|      | venue \                                           |
|------|---------------------------------------------------|
| 0    | M Chinnaswamy Stadium                             |
| 1    | Punjab Cricket Association Stadium, Mohali        |
| 2    | Feroz Shah Kotla                                  |
| 3    | Wankhede Stadium                                  |
| 4    | Eden Gardens                                      |
| ...  | ...                                               |
| 1090 | Rajiv Gandhi International Stadium, Uppal, Hyd... |
| 1091 | Narendra Modi Stadium, Ahmedabad                  |
| 1092 | Narendra Modi Stadium, Ahmedabad                  |
| 1093 | MA Chidambaram Stadium, Chepauk, Chennai          |
| 1094 | MA Chidambaram Stadium, Chepauk, Chennai          |

|      | team1                       | team2 \                     |
|------|-----------------------------|-----------------------------|
| 0    | Royal Challengers Bangalore | Kolkata Knight Riders       |
| 1    | Punjab Kings                | Chennai Super Kings         |
| 2    | Delhi Capitals              | Rajasthan Royals            |
| 3    | Mumbai Indians              | Royal Challengers Bangalore |
| 4    | Kolkata Knight Riders       | Deccan Chargers             |
| ...  | ...                         | ...                         |
| 1090 | Punjab Kings                | Sunrisers Hyderabad         |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders       |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals            |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders       |

|   | toss_winner                 | venue_matches_team1_prior \ |
|---|-----------------------------|-----------------------------|
| 0 | Royal Challengers Bangalore | 0                           |
| 1 | Chennai Super Kings         | 0                           |
| 2 | Rajasthan Royals            | 0                           |
| 3 | Mumbai Indians              | 0                           |

|      |                     |     |     |
|------|---------------------|-----|-----|
| 4    | Deccan Chargers     | ... | 0   |
| ...  | ...                 | ... | ... |
| 1090 | Punjab Kings        | ... | 1   |
| 1091 | Sunrisers Hyderabad | ... | 2   |
| 1092 | Rajasthan Royals    | ... | 1   |
| 1093 | Rajasthan Royals    | ... | 7   |
| 1094 | Sunrisers Hyderabad | ... | 8   |

|      | venue_matches_team2_prior | match_importance | toss_advantage | \ |
|------|---------------------------|------------------|----------------|---|
| 0    | 0                         | League           | 0              |   |
| 1    | 0                         | League           | 1              |   |
| 2    | 0                         | League           | 0              |   |
| 3    | 0                         | League           | 0              |   |
| 4    | 0                         | League           | 0              |   |
| ...  | ...                       | ...              | ...            |   |
| 1090 | 12                        | League           | 0              |   |
| 1091 | 3                         | Playoff          | 0              |   |
| 1092 | 3                         | Playoff          | 1              |   |
| 1093 | 2                         | Playoff          | 0              |   |
| 1094 | 5                         | Final            | 0              |   |

|      | season_match_number | season_total_matches | season_progress | season_phase | \ |
|------|---------------------|----------------------|-----------------|--------------|---|
| 0    | 1                   | 58                   | 0.017241        | Early        |   |
| 1    | 2                   | 58                   | 0.034483        | Early        |   |
| 2    | 3                   | 58                   | 0.051724        | Early        |   |
| 3    | 4                   | 58                   | 0.068966        | Early        |   |
| 4    | 5                   | 58                   | 0.086207        | Early        |   |
| ...  | ...                 | ...                  | ...             | ...          |   |
| 1090 | 67                  | 71                   | 0.943662        | Late         |   |
| 1091 | 68                  | 71                   | 0.957746        | Late         |   |
| 1092 | 69                  | 71                   | 0.971831        | Late         |   |
| 1093 | 70                  | 71                   | 0.985915        | Late         |   |
| 1094 | 71                  | 71                   | 1.000000        | Late         |   |

|      | team1_recent_form | team2_recent_form |
|------|-------------------|-------------------|
| 0    | 0.5               | 0.5               |
| 1    | 0.5               | 0.5               |
| 2    | 0.5               | 0.5               |
| 3    | 0.5               | 0.0               |
| 4    | 1.0               | 0.5               |
| ...  | ...               | ...               |
| 1090 | 0.6               | 0.4               |
| 1091 | 0.6               | 0.8               |
| 1092 | 1.0               | 0.2               |
| 1093 | 0.6               | 0.2               |
| 1094 | 0.6               | 1.0               |

[1095 rows x 33 columns]

```
[21]: ## Head-to-Head-Advantage

from collections import defaultdict

h2h_tracker = defaultdict(lambda: [0,0])

team1_h2h_wins = []
team2_h2h_wins = []

for _, row in df.iterrows():

    t1 = row['team1']
    t2 = row['team2']

    pair = tuple(sorted([t1,t2]))

    wins = h2h_tracker[pair]

    if pair[0] == t1:
        team1_h2h_wins.append(wins[0])
        team2_h2h_wins.append(wins[1])
    else:
        team1_h2h_wins.append(wins[1])
        team2_h2h_wins.append(wins[0])

    if row['winner'] == pair[0]:
        h2h_tracker[pair][0] += 1

    elif row['winner'] == pair[1]:
        h2h_tracker[pair][1] += 1

df['team1_h2h_wins_prior'] = team1_h2h_wins
df['team2_h2h_wins_prior'] = team2_h2h_wins

df
```

```
[21]:
```

|      | id      | season | city       | date       | match_type  | player_of_match | \ |
|------|---------|--------|------------|------------|-------------|-----------------|---|
| 0    | 335982  | 2008   | Bangalore  | 2008-04-18 | League      | BB McCullum     |   |
| 1    | 335983  | 2008   | Chandigarh | 2008-04-19 | League      | MEK Hussey      |   |
| 2    | 335984  | 2008   | Delhi      | 2008-04-19 | League      | MF Maharroof    |   |
| 3    | 335985  | 2008   | Mumbai     | 2008-04-20 | League      | MV Boucher      |   |
| 4    | 335986  | 2008   | Kolkata    | 2008-04-20 | League      | DJ Hussey       |   |
| ...  | ...     | ...    | ...        | ...        | ...         | ...             |   |
| 1090 | 1426307 | 2024   | Hyderabad  | 2024-05-19 | League      | Abhishek Sharma |   |
| 1091 | 1426309 | 2024   | Ahmedabad  | 2024-05-21 | Qualifier 1 | MA Starc        |   |

|      |         |      |           |            |             |               |
|------|---------|------|-----------|------------|-------------|---------------|
| 1092 | 1426310 | 2024 | Ahmedabad | 2024-05-22 | Eliminator  | R Ashwin      |
| 1093 | 1426311 | 2024 | Chennai   | 2024-05-24 | Qualifier 2 | Shahbaz Ahmed |
| 1094 | 1426312 | 2024 | Chennai   | 2024-05-26 | Final       | MA Starc      |

|      | venue \                                           |
|------|---------------------------------------------------|
| 0    | M Chinnaswamy Stadium                             |
| 1    | Punjab Cricket Association Stadium, Mohali        |
| 2    | Feroz Shah Kotla                                  |
| 3    | Wankhede Stadium                                  |
| 4    | Eden Gardens                                      |
| ...  | ...                                               |
| 1090 | Rajiv Gandhi International Stadium, Uppal, Hyd... |
| 1091 | Narendra Modi Stadium, Ahmedabad                  |
| 1092 | Narendra Modi Stadium, Ahmedabad                  |
| 1093 | MA Chidambaram Stadium, Chepauk, Chennai          |
| 1094 | MA Chidambaram Stadium, Chepauk, Chennai          |

|      | team1                       | team2 \                     |
|------|-----------------------------|-----------------------------|
| 0    | Royal Challengers Bangalore | Kolkata Knight Riders       |
| 1    | Punjab Kings                | Chennai Super Kings         |
| 2    | Delhi Capitals              | Rajasthan Royals            |
| 3    | Mumbai Indians              | Royal Challengers Bangalore |
| 4    | Kolkata Knight Riders       | Deccan Chargers             |
| ...  | ...                         | ...                         |
| 1090 | Punjab Kings                | Sunrisers Hyderabad         |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders       |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals            |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders       |

|      | toss_winner                 | ... match_importance | toss_advantage \ |
|------|-----------------------------|----------------------|------------------|
| 0    | Royal Challengers Bangalore | ...                  | League 0         |
| 1    | Chennai Super Kings         | ...                  | League 1         |
| 2    | Rajasthan Royals            | ...                  | League 0         |
| 3    | Mumbai Indians              | ...                  | League 0         |
| 4    | Deccan Chargers             | ...                  | League 0         |
| ...  | ...                         | ...                  | ...              |
| 1090 | Punjab Kings                | ...                  | League 0         |
| 1091 | Sunrisers Hyderabad         | ...                  | Playoff 0        |
| 1092 | Rajasthan Royals            | ...                  | Playoff 1        |
| 1093 | Rajasthan Royals            | ...                  | Playoff 0        |
| 1094 | Sunrisers Hyderabad         | ...                  | Final 0          |

|   | season_match_number | season_total_matches | season_progress | season_phase \ |
|---|---------------------|----------------------|-----------------|----------------|
| 0 | 1                   | 58                   | 0.017241        | Early          |
| 1 | 2                   | 58                   | 0.034483        | Early          |
| 2 | 3                   | 58                   | 0.051724        | Early          |



|      |     |     |          |       |
|------|-----|-----|----------|-------|
| 3    | 4   | 58  | 0.068966 | Early |
| 4    | 5   | 58  | 0.086207 | Early |
| ...  | ... | ... | ...      | ...   |
| 1090 | 67  | 71  | 0.943662 | Late  |
| 1091 | 68  | 71  | 0.957746 | Late  |
| 1092 | 69  | 71  | 0.971831 | Late  |
| 1093 | 70  | 71  | 0.985915 | Late  |
| 1094 | 71  | 71  | 1.000000 | Late  |

|      | team1_recent_form | team2_recent_form | team1_h2h_wins_prior | \ |
|------|-------------------|-------------------|----------------------|---|
| 0    | 0.5               | 0.5               | 0                    |   |
| 1    | 0.5               | 0.5               | 0                    |   |
| 2    | 0.5               | 0.5               | 0                    |   |
| 3    | 0.5               | 0.0               | 0                    |   |
| 4    | 1.0               | 0.5               | 0                    |   |
| ...  | ...               | ...               | ...                  |   |
| 1090 | 0.6               | 0.4               | 7                    |   |
| 1091 | 0.6               | 0.8               | 9                    |   |
| 1092 | 1.0               | 0.2               | 0                    |   |
| 1093 | 0.6               | 0.2               | 10                   |   |
| 1094 | 0.6               | 1.0               | 9                    |   |

|      | team2_h2h_wins_prior |
|------|----------------------|
| 0    | 0                    |
| 1    | 0                    |
| 2    | 0                    |
| 3    | 0                    |
| 4    | 0                    |
| ...  | ...                  |
| 1090 | 15                   |
| 1091 | 17                   |
| 1092 | 1                    |
| 1093 | 9                    |
| 1094 | 18                   |

[1095 rows x 35 columns]

[22]: *## Number of matches each team played before current match*

```

experience = defaultdict(int)

team1_exp = []
team2_exp = []

for _, row in df.iterrows():

    t1 = row['team1']

```

```

t2 = row['team2']

team1_exp.append(experience[t1])
team2_exp.append(experience[t2])

experience[t1] += 1
experience[t2] += 1

df['team1_experience'] = team1_exp
df['team2_experience'] = team2_exp

df['experience_gap'] = (
    df['team1_experience']
    - df['team2_experience']
)

df

```

```

[22]:
      id  season    city    date  match_type  player_of_match \
0    335982   2008  Bangalore  2008-04-18    League    BB McCullum
1    335983   2008  Chandigarh  2008-04-19    League    MEK Hussey
2    335984   2008    Delhi  2008-04-19    League    MF Maharooof
3    335985   2008   Mumbai  2008-04-20    League    MV Boucher
4    335986   2008  Kolkata  2008-04-20    League    DJ Hussey
...
1090  1426307   2024  Hyderabad  2024-05-19    League  Abhishek Sharma
1091  1426309   2024  Ahmedabad  2024-05-21  Qualifier 1    MA Starc
1092  1426310   2024  Ahmedabad  2024-05-22  Eliminator    R Ashwin
1093  1426311   2024    Chennai  2024-05-24  Qualifier 2  Shahbaz Ahmed
1094  1426312   2024    Chennai  2024-05-26    Final    MA Starc

```

```

      venue \
0    M Chinnaswamy Stadium
1    Punjab Cricket Association Stadium, Mohali
2    Feroz Shah Kotla
3    Wankhede Stadium
4    Eden Gardens
...
1090  Rajiv Gandhi International Stadium, Uppal, Hyd...
1091    Narendra Modi Stadium, Ahmedabad
1092    Narendra Modi Stadium, Ahmedabad
1093    MA Chidambaram Stadium, Chepauk, Chennai
1094    MA Chidambaram Stadium, Chepauk, Chennai

```

```

      team1      team2 \
0  Royal Challengers Bangalore  Kolkata Knight Riders
1    Punjab Kings      Chennai Super Kings

```

|      |                             |                             |
|------|-----------------------------|-----------------------------|
| 2    | Delhi Capitals              | Rajasthan Royals            |
| 3    | Mumbai Indians              | Royal Challengers Bangalore |
| 4    | Kolkata Knight Riders       | Deccan Chargers             |
| ...  | ...                         | ...                         |
| 1090 | Punjab Kings                | Sunrisers Hyderabad         |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders       |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals            |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders       |

|      | toss_winner                 | ... | season_total_matches | season_progress | \ |
|------|-----------------------------|-----|----------------------|-----------------|---|
| 0    | Royal Challengers Bangalore | ... | 58                   | 0.017241        |   |
| 1    | Chennai Super Kings         | ... | 58                   | 0.034483        |   |
| 2    | Rajasthan Royals            | ... | 58                   | 0.051724        |   |
| 3    | Mumbai Indians              | ... | 58                   | 0.068966        |   |
| 4    | Deccan Chargers             | ... | 58                   | 0.086207        |   |
| ...  | ...                         | ... | ...                  | ...             |   |
| 1090 | Punjab Kings                | ... | 71                   | 0.943662        |   |
| 1091 | Sunrisers Hyderabad         | ... | 71                   | 0.957746        |   |
| 1092 | Rajasthan Royals            | ... | 71                   | 0.971831        |   |
| 1093 | Rajasthan Royals            | ... | 71                   | 0.985915        |   |
| 1094 | Sunrisers Hyderabad         | ... | 71                   | 1.000000        |   |

|      | season_phase | team1_recent_form | team2_recent_form | team1_h2h_wins_prior | \ |
|------|--------------|-------------------|-------------------|----------------------|---|
| 0    | Early        | 0.5               | 0.5               | 0                    |   |
| 1    | Early        | 0.5               | 0.5               | 0                    |   |
| 2    | Early        | 0.5               | 0.5               | 0                    |   |
| 3    | Early        | 0.5               | 0.0               | 0                    |   |
| 4    | Early        | 1.0               | 0.5               | 0                    |   |
| ...  | ...          | ...               | ...               | ...                  |   |
| 1090 | Late         | 0.6               | 0.4               | 7                    |   |
| 1091 | Late         | 0.6               | 0.8               | 9                    |   |
| 1092 | Late         | 1.0               | 0.2               | 0                    |   |
| 1093 | Late         | 0.6               | 0.2               | 10                   |   |
| 1094 | Late         | 0.6               | 1.0               | 9                    |   |

|      | team2_h2h_wins_prior | team1_experience | team2_experience | experience_gap |
|------|----------------------|------------------|------------------|----------------|
| 0    | 0                    | 0                | 0                | 0              |
| 1    | 0                    | 0                | 0                | 0              |
| 2    | 0                    | 0                | 0                | 0              |
| 3    | 0                    | 0                | 1                | -1             |
| 4    | 0                    | 1                | 0                | 1              |
| ...  | ...                  | ...              | ...              | ...            |
| 1090 | 15                   | 245              | 178              | 67             |
| 1091 | 17                   | 179              | 249              | -70            |
| 1092 | 1                    | 14               | 219              | -205           |
| 1093 | 9                    | 180              | 220              | -40            |

1094 18 181 250 -69

[1095 rows x 38 columns]

```
[23]: ## From Difference
df['recent_form_diff'] = (
    df['team1_recent_form']
    - df['team2_recent_form']
)
```

```
[24]: ## Venue experience difference

df['venue_experience_diff'] = (
    df['venue_matches_team1_prior']
    - df['venue_matches_team2_prior']
)

df
```

```
[24]:
```

|      | id      | season | city       | date       | match_type  | player_of_match \ |
|------|---------|--------|------------|------------|-------------|-------------------|
| 0    | 335982  | 2008   | Bangalore  | 2008-04-18 | League      | BB McCullum       |
| 1    | 335983  | 2008   | Chandigarh | 2008-04-19 | League      | MEK Hussey        |
| 2    | 335984  | 2008   | Delhi      | 2008-04-19 | League      | MF Maharooof      |
| 3    | 335985  | 2008   | Mumbai     | 2008-04-20 | League      | MV Boucher        |
| 4    | 335986  | 2008   | Kolkata    | 2008-04-20 | League      | DJ Hussey         |
| ...  | ...     | ...    | ...        | ...        | ...         | ...               |
| 1090 | 1426307 | 2024   | Hyderabad  | 2024-05-19 | League      | Abhishek Sharma   |
| 1091 | 1426309 | 2024   | Ahmedabad  | 2024-05-21 | Qualifier 1 | MA Starc          |
| 1092 | 1426310 | 2024   | Ahmedabad  | 2024-05-22 | Eliminator  | R Ashwin          |
| 1093 | 1426311 | 2024   | Chennai    | 2024-05-24 | Qualifier 2 | Shahbaz Ahmed     |
| 1094 | 1426312 | 2024   | Chennai    | 2024-05-26 | Final       | MA Starc          |

|      | venue \                                           |
|------|---------------------------------------------------|
| 0    | M Chinnaswamy Stadium                             |
| 1    | Punjab Cricket Association Stadium, Mohali        |
| 2    | Feroz Shah Kotla                                  |
| 3    | Wankhede Stadium                                  |
| 4    | Eden Gardens                                      |
| ...  | ...                                               |
| 1090 | Rajiv Gandhi International Stadium, Uppal, Hyd... |
| 1091 | Narendra Modi Stadium, Ahmedabad                  |
| 1092 | Narendra Modi Stadium, Ahmedabad                  |
| 1093 | MA Chidambaram Stadium, Chepauk, Chennai          |
| 1094 | MA Chidambaram Stadium, Chepauk, Chennai          |

|   | team1                       | team2 \               |
|---|-----------------------------|-----------------------|
| 0 | Royal Challengers Bangalore | Kolkata Knight Riders |

|      |                             |                             |
|------|-----------------------------|-----------------------------|
| 1    | Punjab Kings                | Chennai Super Kings         |
| 2    | Delhi Capitals              | Rajasthan Royals            |
| 3    | Mumbai Indians              | Royal Challengers Bangalore |
| 4    | Kolkata Knight Riders       | Deccan Chargers             |
| ...  | ...                         | ...                         |
| 1090 | Punjab Kings                | Sunrisers Hyderabad         |
| 1091 | Sunrisers Hyderabad         | Kolkata Knight Riders       |
| 1092 | Royal Challengers Bengaluru | Rajasthan Royals            |
| 1093 | Sunrisers Hyderabad         | Rajasthan Royals            |
| 1094 | Sunrisers Hyderabad         | Kolkata Knight Riders       |

|      | toss_winner                 | ... | season_phase | team1_recent_form | \ |
|------|-----------------------------|-----|--------------|-------------------|---|
| 0    | Royal Challengers Bangalore | ... | Early        | 0.5               |   |
| 1    | Chennai Super Kings         | ... | Early        | 0.5               |   |
| 2    | Rajasthan Royals            | ... | Early        | 0.5               |   |
| 3    | Mumbai Indians              | ... | Early        | 0.5               |   |
| 4    | Deccan Chargers             | ... | Early        | 1.0               |   |
| ...  | ...                         | ... | ...          | ...               |   |
| 1090 | Punjab Kings                | ... | Late         | 0.6               |   |
| 1091 | Sunrisers Hyderabad         | ... | Late         | 0.6               |   |
| 1092 | Rajasthan Royals            | ... | Late         | 1.0               |   |
| 1093 | Rajasthan Royals            | ... | Late         | 0.6               |   |
| 1094 | Sunrisers Hyderabad         | ... | Late         | 0.6               |   |

|      | team2_recent_form | team1_h2h_wins_prior | team2_h2h_wins_prior | \ |
|------|-------------------|----------------------|----------------------|---|
| 0    | 0.5               | 0                    | 0                    |   |
| 1    | 0.5               | 0                    | 0                    |   |
| 2    | 0.5               | 0                    | 0                    |   |
| 3    | 0.0               | 0                    | 0                    |   |
| 4    | 0.5               | 0                    | 0                    |   |
| ...  | ...               | ...                  | ...                  |   |
| 1090 | 0.4               | 7                    | 15                   |   |
| 1091 | 0.8               | 9                    | 17                   |   |
| 1092 | 0.2               | 0                    | 1                    |   |
| 1093 | 0.2               | 10                   | 9                    |   |
| 1094 | 1.0               | 9                    | 18                   |   |

|      | team1_experience | team2_experience | experience_gap | recent_form_diff | \ |
|------|------------------|------------------|----------------|------------------|---|
| 0    | 0                | 0                | 0              | 0.0              |   |
| 1    | 0                | 0                | 0              | 0.0              |   |
| 2    | 0                | 0                | 0              | 0.0              |   |
| 3    | 0                | 1                | -1             | 0.5              |   |
| 4    | 1                | 0                | 1              | 0.5              |   |
| ...  | ...              | ...              | ...            | ...              |   |
| 1090 | 245              | 178              | 67             | 0.2              |   |
| 1091 | 179              | 249              | -70            | -0.2             |   |
| 1092 | 14               | 219              | -205           | 0.8              |   |

|      |     |     |     |      |
|------|-----|-----|-----|------|
| 1093 | 180 | 220 | -40 | 0.4  |
| 1094 | 181 | 250 | -69 | -0.4 |

```

venue_experience_diff
0      0
1      0
2      0
3      0
4      0
...
1090   -11
1091    -1
1092    -2
1093     5
1094     3

```

[1095 rows x 40 columns]

## 2.2 Question 2: Team Performance Clustering

### 2.2.1 Task:

Perform **clustering analysis** to group teams based on their performance metrics:

1. Create team-level features:
  - Win percentage
  - Average victory margin (runs/wickets)
  - Toss win to match win ratio
  - Home vs away performance(optional based on assumption)
  - Venue familiarity effect: win% at a team's top-3 most played venues vs other venues.
2. Apply K-means clustering to identify team categories
3. Use elbow method to determine optimal clusters
4. Visualize clusters using PCA for dimensionality reduction(optional)

### 2.2.2 Expected Output:

- Team clusters with labels (e.g., “Dominant Teams”, “Inconsistent Performers”)
- Cluster characteristics interpretation

*# Your code here*

```

[25]: ## Win Percentage

all_teams = pd.concat([
    df['team1'],
    df['team2']
]).unique()

team_stats = []

```

```

for team in all_teams:

    matches_played = (
        ((df['team1']==team) | (df['team2']==team))
    ).sum()

    wins = (df['winner']==team).sum()

    win_pct = wins / matches_played

    team_stats.append({
        'team': team,
        'matches_played': matches_played,
        'wins': wins,
        'win_pct': win_pct
    })

team_features = pd.DataFrame(team_stats)

team_features

```

```

[25]:

```

|    | team                        | matches_played | wins | win_pct  |
|----|-----------------------------|----------------|------|----------|
| 0  | Royal Challengers Bangalore | 240            | 116  | 0.483333 |
| 1  | Punjab Kings                | 246            | 112  | 0.455285 |
| 2  | Delhi Capitals              | 252            | 115  | 0.456349 |
| 3  | Mumbai Indians              | 261            | 144  | 0.551724 |
| 4  | Kolkata Knight Riders       | 251            | 131  | 0.521912 |
| 5  | Rajasthan Royals            | 221            | 112  | 0.506787 |
| 6  | Deccan Chargers             | 75             | 29   | 0.386667 |
| 7  | Chennai Super Kings         | 238            | 138  | 0.579832 |
| 8  | Kochi Tuskers Kerala        | 14             | 6    | 0.428571 |
| 9  | Pune Warriors               | 46             | 12   | 0.260870 |
| 10 | Sunrisers Hyderabad         | 182            | 88   | 0.483516 |
| 11 | Gujarat Lions               | 30             | 13   | 0.433333 |
| 12 | Rising Pune Supergiants     | 14             | 5    | 0.357143 |
| 13 | Rising Pune Supergiant      | 16             | 10   | 0.625000 |
| 14 | Lucknow Super Giants        | 44             | 24   | 0.545455 |
| 15 | Gujarat Titans              | 45             | 28   | 0.622222 |
| 16 | Royal Challengers Bengaluru | 15             | 7    | 0.466667 |

```

[26]: ## Average Victory margin, this needs to be looked at from runs and wickets_
      ↪perspective
      ## Average runs margin
run_margin = (
    df[df['result']=='runs']
    .groupby('winner')['result_margin']

```

```

        .mean()
    )

    ## Average wickets margin

    wicket_margin = (
        df[df['result']=='wickets']
        .groupby('winner')['result_margin']
        .mean()
    )
    ## Merge run_margin and wicket_margin

    team_features['avg_run_margin'] = (
        team_features['team']
        .map(run_margin)
    )

    team_features['avg_wicket_margin'] = (
        team_features['team']
        .map(wicket_margin)
    )

    team_features.fillna(0, inplace=True)

    team_features

```

```

[26]:
      team  matches_played  wins  win_pct  \
0  Royal Challengers Bangalore      240    116  0.483333
1      Punjab Kings      246    112  0.455285
2      Delhi Capitals      252    115  0.456349
3      Mumbai Indians      261    144  0.551724
4  Kolkata Knight Riders      251    131  0.521912
5      Rajasthan Royals      221    112  0.506787
6      Deccan Chargers       75     29  0.386667
7      Chennai Super Kings      238    138  0.579832
8      Kochi Tuskers Kerala       14     6  0.428571
9      Pune Warriors         46    12  0.260870
10     Sunrisers Hyderabad      182    88  0.483516
11     Gujarat Lions         30    13  0.433333
12     Rising Pune Supergiants      14     5  0.357143
13     Rising Pune Supergiant      16    10  0.625000
14     Lucknow Super Giants       44    24  0.545455
15     Gujarat Titans         45    28  0.622222
16  Royal Challengers Bengaluru      15     7  0.466667

      avg_run_margin  avg_wicket_margin
0      33.924528      6.573770

```



|    |           |          |
|----|-----------|----------|
| 1  | 24.784314 | 6.206897 |
| 2  | 25.630435 | 6.227273 |
| 3  | 32.971831 | 6.197183 |
| 4  | 33.592593 | 6.197368 |
| 5  | 30.837209 | 5.835821 |
| 6  | 23.388889 | 6.545455 |
| 7  | 34.943662 | 6.029851 |
| 8  | 11.500000 | 7.500000 |
| 9  | 23.166667 | 6.000000 |
| 10 | 24.840909 | 6.720930 |
| 11 | 1.000000  | 5.416667 |
| 12 | 26.500000 | 6.666667 |
| 13 | 24.600000 | 6.200000 |
| 14 | 24.437500 | 5.375000 |
| 15 | 34.181818 | 5.764706 |
| 16 | 42.250000 | 5.666667 |

```
[27]: ## Toss win to match win ratio
ratio_data = []

for team in all_teams:

    toss_wins = (
        df['toss_winner']==team
    ).sum()

    toss_and_match_wins = (
        (df['toss_winner']==team)
        &
        (df['winner']==team)
    ).sum()

    ratio = (
        toss_and_match_wins/toss_wins
        if toss_wins > 0
        else 0
    )

    ratio_data.append(ratio)

team_features['toss_win_conversion_ratio'] = ratio_data
team_features
```

```
[27]:
```

|   | team                        | matches_played | wins | win_pct  | \ |
|---|-----------------------------|----------------|------|----------|---|
| 0 | Royal Challengers Bangalore | 240            | 116  | 0.483333 |   |
| 1 | Punjab Kings                | 246            | 112  | 0.455285 |   |
| 2 | Delhi Capitals              | 252            | 115  | 0.456349 |   |

|    |                             |     |     |          |
|----|-----------------------------|-----|-----|----------|
| 3  | Mumbai Indians              | 261 | 144 | 0.551724 |
| 4  | Kolkata Knight Riders       | 251 | 131 | 0.521912 |
| 5  | Rajasthan Royals            | 221 | 112 | 0.506787 |
| 6  | Deccan Chargers             | 75  | 29  | 0.386667 |
| 7  | Chennai Super Kings         | 238 | 138 | 0.579832 |
| 8  | Kochi Tuskers Kerala        | 14  | 6   | 0.428571 |
| 9  | Pune Warriors               | 46  | 12  | 0.260870 |
| 10 | Sunrisers Hyderabad         | 182 | 88  | 0.483516 |
| 11 | Gujarat Lions               | 30  | 13  | 0.433333 |
| 12 | Rising Pune Supergiants     | 14  | 5   | 0.357143 |
| 13 | Rising Pune Supergiant      | 16  | 10  | 0.625000 |
| 14 | Lucknow Super Giants        | 44  | 24  | 0.545455 |
| 15 | Gujarat Titans              | 45  | 28  | 0.622222 |
| 16 | Royal Challengers Bengaluru | 15  | 7   | 0.466667 |

|    | avg_run_margin | avg_wicket_margin | toss_win_conversion_ratio |
|----|----------------|-------------------|---------------------------|
| 0  | 33.924528      | 6.573770          | 0.504425                  |
| 1  | 24.784314      | 6.206897          | 0.412844                  |
| 2  | 25.630435      | 6.227273          | 0.469231                  |
| 3  | 32.971831      | 6.197183          | 0.545455                  |
| 4  | 33.592593      | 6.197368          | 0.557377                  |
| 5  | 30.837209      | 5.835821          | 0.500000                  |
| 6  | 23.388889      | 6.545455          | 0.441860                  |
| 7  | 34.943662      | 6.029851          | 0.614754                  |
| 8  | 11.500000      | 7.500000          | 0.500000                  |
| 9  | 23.166667      | 6.000000          | 0.150000                  |
| 10 | 24.840909      | 6.720930          | 0.431818                  |
| 11 | 1.000000       | 5.416667          | 0.666667                  |
| 12 | 26.500000      | 6.666667          | 0.428571                  |
| 13 | 24.600000      | 6.200000          | 0.833333                  |
| 14 | 24.437500      | 5.375000          | 0.526316                  |
| 15 | 34.181818      | 5.764706          | 0.636364                  |
| 16 | 42.250000      | 5.666667          | 0.500000                  |

```
[28]: ## Home vs Away Performance

## Home Win %

home_wins = []

for team in all_teams:

    home_matches = df[
        (
            (df['team1']==team)
            &
            (df['home_advantage_team1']==1)
        )
    ]
```

```

    )
    |
    (
        (df['team2']==team)
        &
        (df['home_advantage_team2']==1)
    )
]

home_matches_count = len(home_matches)

wins = (
    home_matches['winner']==team
).sum()

pct = (
    wins/home_matches_count
    if home_matches_count > 0
    else 0
)

home_wins.append(pct)

team_features['home_win_pct'] = home_wins

## Away win %

away_wins = []

for team in all_teams:

    away_matches = df[
        (
            (df['team1']==team)
            &
            (df['home_advantage_team1']==0)
        )
        |
        (
            (df['team2']==team)
            &
            (df['home_advantage_team2']==0)
        )
    ]

    total = len(away_matches)

```

```

wins = (
    away_matches['winner']==team
).sum()

pct = wins/total if total > 0 else 0

away_wins.append(pct)

team_features['away_win_pct'] = away_wins

team_features

```

```

[28]:

```

|    | team                        | matches_played | wins | win_pct  | \ |
|----|-----------------------------|----------------|------|----------|---|
| 0  | Royal Challengers Bangalore | 240            | 116  | 0.483333 |   |
| 1  | Punjab Kings                | 246            | 112  | 0.455285 |   |
| 2  | Delhi Capitals              | 252            | 115  | 0.456349 |   |
| 3  | Mumbai Indians              | 261            | 144  | 0.551724 |   |
| 4  | Kolkata Knight Riders       | 251            | 131  | 0.521912 |   |
| 5  | Rajasthan Royals            | 221            | 112  | 0.506787 |   |
| 6  | Deccan Chargers             | 75             | 29   | 0.386667 |   |
| 7  | Chennai Super Kings         | 238            | 138  | 0.579832 |   |
| 8  | Kochi Tuskers Kerala        | 14             | 6    | 0.428571 |   |
| 9  | Pune Warriors               | 46             | 12   | 0.260870 |   |
| 10 | Sunrisers Hyderabad         | 182            | 88   | 0.483516 |   |
| 11 | Gujarat Lions               | 30             | 13   | 0.433333 |   |
| 12 | Rising Pune Supergiants     | 14             | 5    | 0.357143 |   |
| 13 | Rising Pune Supergiant      | 16             | 10   | 0.625000 |   |
| 14 | Lucknow Super Giants        | 44             | 24   | 0.545455 |   |
| 15 | Gujarat Titans              | 45             | 28   | 0.622222 |   |
| 16 | Royal Challengers Bengaluru | 15             | 7    | 0.466667 |   |

|    | avg_run_margin | avg_wicket_margin | toss_win_conversion_ratio | \ |
|----|----------------|-------------------|---------------------------|---|
| 0  | 33.924528      | 6.573770          | 0.504425                  |   |
| 1  | 24.784314      | 6.206897          | 0.412844                  |   |
| 2  | 25.630435      | 6.227273          | 0.469231                  |   |
| 3  | 32.971831      | 6.197183          | 0.545455                  |   |
| 4  | 33.592593      | 6.197368          | 0.557377                  |   |
| 5  | 30.837209      | 5.835821          | 0.500000                  |   |
| 6  | 23.388889      | 6.545455          | 0.441860                  |   |
| 7  | 34.943662      | 6.029851          | 0.614754                  |   |
| 8  | 11.500000      | 7.500000          | 0.500000                  |   |
| 9  | 23.166667      | 6.000000          | 0.150000                  |   |
| 10 | 24.840909      | 6.720930          | 0.431818                  |   |
| 11 | 1.000000       | 5.416667          | 0.666667                  |   |
| 12 | 26.500000      | 6.666667          | 0.428571                  |   |
| 13 | 24.600000      | 6.200000          | 0.833333                  |   |
| 14 | 24.437500      | 5.375000          | 0.526316                  |   |

|    |           |          |          |
|----|-----------|----------|----------|
| 15 | 34.181818 | 5.764706 | 0.636364 |
| 16 | 42.250000 | 5.666667 | 0.500000 |

|    | home_win_pct | away_win_pct |
|----|--------------|--------------|
| 0  | 0.000000     | 0.483333     |
| 1  | 0.508197     | 0.437838     |
| 2  | 0.451220     | 0.458824     |
| 3  | 0.615385     | 0.509554     |
| 4  | 0.590909     | 0.484663     |
| 5  | 0.649123     | 0.457317     |
| 6  | 0.000000     | 0.386667     |
| 7  | 0.704225     | 0.526946     |
| 8  | 0.000000     | 0.428571     |
| 9  | 0.000000     | 0.260870     |
| 10 | 0.631579     | 0.416000     |
| 11 | 0.000000     | 0.433333     |
| 12 | 0.000000     | 0.357143     |
| 13 | 0.000000     | 0.625000     |
| 14 | 0.500000     | 0.566667     |
| 15 | 0.562500     | 0.655172     |
| 16 | 0.000000     | 0.466667     |

```
[29]: ## Venue Familiarity Effect
      ## Comparing win % at the top-3 most played venues vs win % every where else

venue_effect = []

for team in all_teams:

    team_matches = df[
        (df['team1']==team)
        |
        (df['team2']==team)
    ]

    top3_venues = (
        team_matches['venue']
        .value_counts()
        .head(3)
        .index
    )

    top3_matches = team_matches[
        team_matches['venue'].isin(top3_venues)
    ]

    other_matches = team_matches[
```

```

~team_matches['venue'].isin(top3_venues)
]

top3_win_pct = (
    (top3_matches['winner']==team).mean()
)

other_win_pct = (
    (other_matches['winner']==team).mean()
)

venue_effect.append(
    top3_win_pct - other_win_pct
)

team_features['venue_familiarity_effect'] = venue_effect

team_features

```

```

[29]:

```

|    | team                        | matches_played | wins | win_pct  | \ |
|----|-----------------------------|----------------|------|----------|---|
| 0  | Royal Challengers Bangalore | 240            | 116  | 0.483333 |   |
| 1  | Punjab Kings                | 246            | 112  | 0.455285 |   |
| 2  | Delhi Capitals              | 252            | 115  | 0.456349 |   |
| 3  | Mumbai Indians              | 261            | 144  | 0.551724 |   |
| 4  | Kolkata Knight Riders       | 251            | 131  | 0.521912 |   |
| 5  | Rajasthan Royals            | 221            | 112  | 0.506787 |   |
| 6  | Deccan Chargers             | 75             | 29   | 0.386667 |   |
| 7  | Chennai Super Kings         | 238            | 138  | 0.579832 |   |
| 8  | Kochi Tuskers Kerala        | 14             | 6    | 0.428571 |   |
| 9  | Pune Warriors               | 46             | 12   | 0.260870 |   |
| 10 | Sunrisers Hyderabad         | 182            | 88   | 0.483516 |   |
| 11 | Gujarat Lions               | 30             | 13   | 0.433333 |   |
| 12 | Rising Pune Supergiants     | 14             | 5    | 0.357143 |   |
| 13 | Rising Pune Supergiant      | 16             | 10   | 0.625000 |   |
| 14 | Lucknow Super Giants        | 44             | 24   | 0.545455 |   |
| 15 | Gujarat Titans              | 45             | 28   | 0.622222 |   |
| 16 | Royal Challengers Bengaluru | 15             | 7    | 0.466667 |   |

|   | avg_run_margin | avg_wicket_margin | toss_win_conversion_ratio | \ |
|---|----------------|-------------------|---------------------------|---|
| 0 | 33.924528      | 6.573770          | 0.504425                  |   |
| 1 | 24.784314      | 6.206897          | 0.412844                  |   |
| 2 | 25.630435      | 6.227273          | 0.469231                  |   |
| 3 | 32.971831      | 6.197183          | 0.545455                  |   |
| 4 | 33.592593      | 6.197368          | 0.557377                  |   |
| 5 | 30.837209      | 5.835821          | 0.500000                  |   |
| 6 | 23.388889      | 6.545455          | 0.441860                  |   |
| 7 | 34.943662      | 6.029851          | 0.614754                  |   |

|    |           |          |          |
|----|-----------|----------|----------|
| 8  | 11.500000 | 7.500000 | 0.500000 |
| 9  | 23.166667 | 6.000000 | 0.150000 |
| 10 | 24.840909 | 6.720930 | 0.431818 |
| 11 | 1.000000  | 5.416667 | 0.666667 |
| 12 | 26.500000 | 6.666667 | 0.428571 |
| 13 | 24.600000 | 6.200000 | 0.833333 |
| 14 | 24.437500 | 5.375000 | 0.526316 |
| 15 | 34.181818 | 5.764706 | 0.636364 |
| 16 | 42.250000 | 5.666667 | 0.500000 |

|    | home_win_pct | away_win_pct | venue_familiarity_effect |
|----|--------------|--------------|--------------------------|
| 0  | 0.000000     | 0.483333     | 0.044258                 |
| 1  | 0.508197     | 0.437838     | 0.001114                 |
| 2  | 0.451220     | 0.458824     | -0.014019                |
| 3  | 0.615385     | 0.509554     | 0.129586                 |
| 4  | 0.590909     | 0.484663     | 0.105595                 |
| 5  | 0.649123     | 0.457317     | 0.187136                 |
| 6  | 0.000000     | 0.386667     | -0.218085                |
| 7  | 0.704225     | 0.526946     | 0.153346                 |
| 8  | 0.000000     | 0.428571     | -0.125000                |
| 9  | 0.000000     | 0.260870     | 0.041905                 |
| 10 | 0.631579     | 0.416000     | 0.241123                 |
| 11 | 0.000000     | 0.433333     | -0.185520                |
| 12 | 0.000000     | 0.357143     | 0.041667                 |
| 13 | 0.000000     | 0.625000     | 0.327273                 |
| 14 | 0.500000     | 0.566667     | 0.041408                 |
| 15 | 0.562500     | 0.655172     | -0.140000                |
| 16 | 0.000000     | 0.466667     | 0.100000                 |

```
[30]: ## Recent era win %

recent_seasons = [2020, 2021, 2022, 2023, 2024]

recent_df = df[df['season'].isin(recent_seasons)]

recent_win_pct = {}

for team in all_teams:

    matches = recent_df[
        (recent_df['team1']==team) |
        (recent_df['team2']==team)
    ]

    if len(matches) > 0:

        wins = (matches['winner']==team).sum()
```

```

recent_win_pct[team] = wins/len(matches)

else:
    recent_win_pct[team] = 0

team_features['recent_win_pct'] = (
    team_features['team']
    .map(recent_win_pct)
)
team_features

```

```

[30]:
      team  matches_played  wins  win_pct \
0  Royal Challengers Bangalore      240   116  0.483333
1      Punjab Kings      246   112  0.455285
2      Delhi Capitals      252   115  0.456349
3      Mumbai Indians      261   144  0.551724
4  Kolkata Knight Riders      251   131  0.521912
5      Rajasthan Royals      221   112  0.506787
6      Deccan Chargers       75    29  0.386667
7      Chennai Super Kings      238   138  0.579832
8      Kochi Tuskers Kerala      14     6  0.428571
9      Pune Warriors         46    12  0.260870
10     Sunrisers Hyderabad      182   88  0.483516
11     Gujarat Lions          30    13  0.433333
12     Rising Pune Supergiants     14     5  0.357143
13     Rising Pune Supergiant     16    10  0.625000
14     Lucknow Super Giants       44    24  0.545455
15     Gujarat Titans          45    28  0.622222
16  Royal Challengers Bengaluru     15     7  0.466667

      avg_run_margin  avg_wicket_margin  toss_win_conversion_ratio \
0      33.924528      6.573770      0.504425
1      24.784314      6.206897      0.412844
2      25.630435      6.227273      0.469231
3      32.971831      6.197183      0.545455
4      33.592593      6.197368      0.557377
5      30.837209      5.835821      0.500000
6      23.388889      6.545455      0.441860
7      34.943662      6.029851      0.614754
8      11.500000      7.500000      0.500000
9      23.166667      6.000000      0.150000
10     24.840909      6.720930      0.431818
11      1.000000      5.416667      0.666667
12     26.500000      6.666667      0.428571
13     24.600000      6.200000      0.833333
14     24.437500      5.375000      0.526316

```



|    |           |          |          |
|----|-----------|----------|----------|
| 15 | 34.181818 | 5.764706 | 0.636364 |
| 16 | 42.250000 | 5.666667 | 0.500000 |

|    | home_win_pct | away_win_pct | venue_familiarity_effect | recent_win_pct |
|----|--------------|--------------|--------------------------|----------------|
| 0  | 0.000000     | 0.483333     | 0.044258                 | 0.533333       |
| 1  | 0.508197     | 0.437838     | 0.001114                 | 0.428571       |
| 2  | 0.451220     | 0.458824     | -0.014019                | 0.506667       |
| 3  | 0.615385     | 0.509554     | 0.129586                 | 0.472973       |
| 4  | 0.590909     | 0.484663     | 0.105595                 | 0.534247       |
| 5  | 0.649123     | 0.457317     | 0.187136                 | 0.500000       |
| 6  | 0.000000     | 0.386667     | -0.218085                | 0.000000       |
| 7  | 0.704225     | 0.526946     | 0.153346                 | 0.513514       |
| 8  | 0.000000     | 0.428571     | -0.125000                | 0.000000       |
| 9  | 0.000000     | 0.260870     | 0.041905                 | 0.000000       |
| 10 | 0.631579     | 0.416000     | 0.241123                 | 0.405405       |
| 11 | 0.000000     | 0.433333     | -0.185520                | 0.000000       |
| 12 | 0.000000     | 0.357143     | 0.041667                 | 0.000000       |
| 13 | 0.000000     | 0.625000     | 0.327273                 | 0.000000       |
| 14 | 0.500000     | 0.566667     | 0.041408                 | 0.545455       |
| 15 | 0.562500     | 0.655172     | -0.140000                | 0.622222       |
| 16 | 0.000000     | 0.466667     | 0.100000                 | 0.466667       |

```
[31]: ## Playoff Appearance Rate

playoff_df = df[
    df['match_importance'].isin(
        ['Playoff', 'Final']
    )
]

playoff_rate = {}

for team in all_teams:

    playoff_matches = (
        (playoff_df['team1']==team) |
        (playoff_df['team2']==team)
    ).sum()

    total_matches = (
        (df['team1']==team) |
        (df['team2']==team)
    ).sum()

    playoff_rate[team] = (
        playoff_matches/total_matches
        if total_matches > 0
```

```

        else 0
    )

team_features['playoff_rate'] = (
    team_features['team']
    .map(playoff_rate)
)

team_features

```

```

[31]:
      team  matches_played  wins  win_pct \
0  Royal Challengers Bangalore      240   116  0.483333
1      Punjab Kings      246   112  0.455285
2      Delhi Capitals      252   115  0.456349
3      Mumbai Indians      261   144  0.551724
4  Kolkata Knight Riders      251   131  0.521912
5      Rajasthan Royals      221   112  0.506787
6      Deccan Chargers       75    29  0.386667
7      Chennai Super Kings      238   138  0.579832
8      Kochi Tuskers Kerala      14     6  0.428571
9      Pune Warriors         46    12  0.260870
10     Sunrisers Hyderabad      182    88  0.483516
11     Gujarat Lions          30    13  0.433333
12     Rising Pune Supergiants     14     5  0.357143
13     Rising Pune Supergiant     16    10  0.625000
14     Lucknow Super Giants       44    24  0.545455
15     Gujarat Titans           45    28  0.622222
16  Royal Challengers Bengaluru     15     7  0.466667

      avg_run_margin  avg_wicket_margin  toss_win_conversion_ratio \
0      33.924528      6.573770      0.504425
1      24.784314      6.206897      0.412844
2      25.630435      6.227273      0.469231
3      32.971831      6.197183      0.545455
4      33.592593      6.197368      0.557377
5      30.837209      5.835821      0.500000
6      23.388889      6.545455      0.441860
7      34.943662      6.029851      0.614754
8      11.500000      7.500000      0.500000
9      23.166667      6.000000      0.150000
10     24.840909      6.720930      0.431818
11      1.000000      5.416667      0.666667
12     26.500000      6.666667      0.428571
13     24.600000      6.200000      0.833333
14     24.437500      5.375000      0.526316
15     34.181818      5.764706      0.636364
16     42.250000      5.666667      0.500000

```

|    | home_win_pct | away_win_pct | venue_familiarity_effect | recent_win_pct | \ |
|----|--------------|--------------|--------------------------|----------------|---|
| 0  | 0.000000     | 0.483333     | 0.044258                 | 0.533333       |   |
| 1  | 0.508197     | 0.437838     | 0.001114                 | 0.428571       |   |
| 2  | 0.451220     | 0.458824     | -0.014019                | 0.506667       |   |
| 3  | 0.615385     | 0.509554     | 0.129586                 | 0.472973       |   |
| 4  | 0.590909     | 0.484663     | 0.105595                 | 0.534247       |   |
| 5  | 0.649123     | 0.457317     | 0.187136                 | 0.500000       |   |
| 6  | 0.000000     | 0.386667     | -0.218085                | 0.000000       |   |
| 7  | 0.704225     | 0.526946     | 0.153346                 | 0.513514       |   |
| 8  | 0.000000     | 0.428571     | -0.125000                | 0.000000       |   |
| 9  | 0.000000     | 0.260870     | 0.041905                 | 0.000000       |   |
| 10 | 0.631579     | 0.416000     | 0.241123                 | 0.405405       |   |
| 11 | 0.000000     | 0.433333     | -0.185520                | 0.000000       |   |
| 12 | 0.000000     | 0.357143     | 0.041667                 | 0.000000       |   |
| 13 | 0.000000     | 0.625000     | 0.327273                 | 0.000000       |   |
| 14 | 0.500000     | 0.566667     | 0.041408                 | 0.545455       |   |
| 15 | 0.562500     | 0.655172     | -0.140000                | 0.622222       |   |
| 16 | 0.000000     | 0.466667     | 0.100000                 | 0.466667       |   |

|    | playoff_rate |
|----|--------------|
| 0  | 0.062500     |
| 1  | 0.016260     |
| 2  | 0.043651     |
| 3  | 0.076628     |
| 4  | 0.059761     |
| 5  | 0.049774     |
| 6  | 0.053333     |
| 7  | 0.109244     |
| 8  | 0.000000     |
| 9  | 0.000000     |
| 10 | 0.076923     |
| 11 | 0.066667     |
| 12 | 0.000000     |
| 13 | 0.125000     |
| 14 | 0.045455     |
| 15 | 0.111111     |
| 16 | 0.066667     |

[32]: *## Dominance Score*

```
team_features['dominance_score'] = (
    team_features['avg_run_margin']*0.6
    +
    team_features['avg_wicket_margin']*0.4
)
```

```
team_features
```

[32]:

|    | team                        | matches_played | wins | win_pct  | \ |
|----|-----------------------------|----------------|------|----------|---|
| 0  | Royal Challengers Bangalore | 240            | 116  | 0.483333 |   |
| 1  | Punjab Kings                | 246            | 112  | 0.455285 |   |
| 2  | Delhi Capitals              | 252            | 115  | 0.456349 |   |
| 3  | Mumbai Indians              | 261            | 144  | 0.551724 |   |
| 4  | Kolkata Knight Riders       | 251            | 131  | 0.521912 |   |
| 5  | Rajasthan Royals            | 221            | 112  | 0.506787 |   |
| 6  | Deccan Chargers             | 75             | 29   | 0.386667 |   |
| 7  | Chennai Super Kings         | 238            | 138  | 0.579832 |   |
| 8  | Kochi Tuskers Kerala        | 14             | 6    | 0.428571 |   |
| 9  | Pune Warriors               | 46             | 12   | 0.260870 |   |
| 10 | Sunrisers Hyderabad         | 182            | 88   | 0.483516 |   |
| 11 | Gujarat Lions               | 30             | 13   | 0.433333 |   |
| 12 | Rising Pune Supergiants     | 14             | 5    | 0.357143 |   |
| 13 | Rising Pune Supergiant      | 16             | 10   | 0.625000 |   |
| 14 | Lucknow Super Giants        | 44             | 24   | 0.545455 |   |
| 15 | Gujarat Titans              | 45             | 28   | 0.622222 |   |
| 16 | Royal Challengers Bengaluru | 15             | 7    | 0.466667 |   |

|    | avg_run_margin | avg_wicket_margin | toss_win_conversion_ratio | \ |
|----|----------------|-------------------|---------------------------|---|
| 0  | 33.924528      | 6.573770          | 0.504425                  |   |
| 1  | 24.784314      | 6.206897          | 0.412844                  |   |
| 2  | 25.630435      | 6.227273          | 0.469231                  |   |
| 3  | 32.971831      | 6.197183          | 0.545455                  |   |
| 4  | 33.592593      | 6.197368          | 0.557377                  |   |
| 5  | 30.837209      | 5.835821          | 0.500000                  |   |
| 6  | 23.388889      | 6.545455          | 0.441860                  |   |
| 7  | 34.943662      | 6.029851          | 0.614754                  |   |
| 8  | 11.500000      | 7.500000          | 0.500000                  |   |
| 9  | 23.166667      | 6.000000          | 0.150000                  |   |
| 10 | 24.840909      | 6.720930          | 0.431818                  |   |
| 11 | 1.000000       | 5.416667          | 0.666667                  |   |
| 12 | 26.500000      | 6.666667          | 0.428571                  |   |
| 13 | 24.600000      | 6.200000          | 0.833333                  |   |
| 14 | 24.437500      | 5.375000          | 0.526316                  |   |
| 15 | 34.181818      | 5.764706          | 0.636364                  |   |
| 16 | 42.250000      | 5.666667          | 0.500000                  |   |

|   | home_win_pct | away_win_pct | venue_familiarity_effect | recent_win_pct | \ |
|---|--------------|--------------|--------------------------|----------------|---|
| 0 | 0.000000     | 0.483333     | 0.044258                 | 0.533333       |   |
| 1 | 0.508197     | 0.437838     | 0.001114                 | 0.428571       |   |
| 2 | 0.451220     | 0.458824     | -0.014019                | 0.506667       |   |
| 3 | 0.615385     | 0.509554     | 0.129586                 | 0.472973       |   |
| 4 | 0.590909     | 0.484663     | 0.105595                 | 0.534247       |   |
| 5 | 0.649123     | 0.457317     | 0.187136                 | 0.500000       |   |

|    |          |          |           |          |
|----|----------|----------|-----------|----------|
| 6  | 0.000000 | 0.386667 | -0.218085 | 0.000000 |
| 7  | 0.704225 | 0.526946 | 0.153346  | 0.513514 |
| 8  | 0.000000 | 0.428571 | -0.125000 | 0.000000 |
| 9  | 0.000000 | 0.260870 | 0.041905  | 0.000000 |
| 10 | 0.631579 | 0.416000 | 0.241123  | 0.405405 |
| 11 | 0.000000 | 0.433333 | -0.185520 | 0.000000 |
| 12 | 0.000000 | 0.357143 | 0.041667  | 0.000000 |
| 13 | 0.000000 | 0.625000 | 0.327273  | 0.000000 |
| 14 | 0.500000 | 0.566667 | 0.041408  | 0.545455 |
| 15 | 0.562500 | 0.655172 | -0.140000 | 0.622222 |
| 16 | 0.000000 | 0.466667 | 0.100000  | 0.466667 |

|    | playoff_rate | dominance_score |
|----|--------------|-----------------|
| 0  | 0.062500     | 22.984225       |
| 1  | 0.016260     | 17.353347       |
| 2  | 0.043651     | 17.869170       |
| 3  | 0.076628     | 22.261972       |
| 4  | 0.059761     | 22.634503       |
| 5  | 0.049774     | 20.836654       |
| 6  | 0.053333     | 16.651515       |
| 7  | 0.109244     | 23.378137       |
| 8  | 0.000000     | 9.900000        |
| 9  | 0.000000     | 16.300000       |
| 10 | 0.076923     | 17.592918       |
| 11 | 0.066667     | 2.766667        |
| 12 | 0.000000     | 18.566667       |
| 13 | 0.125000     | 17.240000       |
| 14 | 0.045455     | 16.812500       |
| 15 | 0.111111     | 22.814973       |
| 16 | 0.066667     | 27.616667       |

```
[33]: ## Recent From Stability

season_win_rates = []

for team in all_teams:

    rates = []

    for season in sorted(df['season'].unique()):

        season_matches = df[
            (
                (df['team1']==team)
                |
                (df['team2']==team)
            )
        ]
```

```

        &
        (df['season']==season)
    ]

    if len(season_matches) > 0:

        win_pct = (
            (season_matches['winner']==team)
            .mean()
        )

        rates.append(win_pct)

    season_win_rates.append(
        np.std(rates)
        if len(rates)>1
        else 0
    )

team_features['performance_volatility'] = season_win_rates

team_features

```

```

[33]:

```

|    | team                        | matches_played | wins | win_pct  | \ |
|----|-----------------------------|----------------|------|----------|---|
| 0  | Royal Challengers Bangalore | 240            | 116  | 0.483333 |   |
| 1  | Punjab Kings                | 246            | 112  | 0.455285 |   |
| 2  | Delhi Capitals              | 252            | 115  | 0.456349 |   |
| 3  | Mumbai Indians              | 261            | 144  | 0.551724 |   |
| 4  | Kolkata Knight Riders       | 251            | 131  | 0.521912 |   |
| 5  | Rajasthan Royals            | 221            | 112  | 0.506787 |   |
| 6  | Deccan Chargers             | 75             | 29   | 0.386667 |   |
| 7  | Chennai Super Kings         | 238            | 138  | 0.579832 |   |
| 8  | Kochi Tuskers Kerala        | 14             | 6    | 0.428571 |   |
| 9  | Pune Warriors               | 46             | 12   | 0.260870 |   |
| 10 | Sunrisers Hyderabad         | 182            | 88   | 0.483516 |   |
| 11 | Gujarat Lions               | 30             | 13   | 0.433333 |   |
| 12 | Rising Pune Supergiants     | 14             | 5    | 0.357143 |   |
| 13 | Rising Pune Supergiant      | 16             | 10   | 0.625000 |   |
| 14 | Lucknow Super Giants        | 44             | 24   | 0.545455 |   |
| 15 | Gujarat Titans              | 45             | 28   | 0.622222 |   |
| 16 | Royal Challengers Bengaluru | 15             | 7    | 0.466667 |   |

|   | avg_run_margin | avg_wicket_margin | toss_win_conversion_ratio | \ |
|---|----------------|-------------------|---------------------------|---|
| 0 | 33.924528      | 6.573770          | 0.504425                  |   |
| 1 | 24.784314      | 6.206897          | 0.412844                  |   |
| 2 | 25.630435      | 6.227273          | 0.469231                  |   |
| 3 | 32.971831      | 6.197183          | 0.545455                  |   |

|    |           |          |          |
|----|-----------|----------|----------|
| 4  | 33.592593 | 6.197368 | 0.557377 |
| 5  | 30.837209 | 5.835821 | 0.500000 |
| 6  | 23.388889 | 6.545455 | 0.441860 |
| 7  | 34.943662 | 6.029851 | 0.614754 |
| 8  | 11.500000 | 7.500000 | 0.500000 |
| 9  | 23.166667 | 6.000000 | 0.150000 |
| 10 | 24.840909 | 6.720930 | 0.431818 |
| 11 | 1.000000  | 5.416667 | 0.666667 |
| 12 | 26.500000 | 6.666667 | 0.428571 |
| 13 | 24.600000 | 6.200000 | 0.833333 |
| 14 | 24.437500 | 5.375000 | 0.526316 |
| 15 | 34.181818 | 5.764706 | 0.636364 |
| 16 | 42.250000 | 5.666667 | 0.500000 |

|    | home_win_pct | away_win_pct | venue_familiarity_effect | recent_win_pct \ |
|----|--------------|--------------|--------------------------|------------------|
| 0  | 0.000000     | 0.483333     | 0.044258                 | 0.533333         |
| 1  | 0.508197     | 0.437838     | 0.001114                 | 0.428571         |
| 2  | 0.451220     | 0.458824     | -0.014019                | 0.506667         |
| 3  | 0.615385     | 0.509554     | 0.129586                 | 0.472973         |
| 4  | 0.590909     | 0.484663     | 0.105595                 | 0.534247         |
| 5  | 0.649123     | 0.457317     | 0.187136                 | 0.500000         |
| 6  | 0.000000     | 0.386667     | -0.218085                | 0.000000         |
| 7  | 0.704225     | 0.526946     | 0.153346                 | 0.513514         |
| 8  | 0.000000     | 0.428571     | -0.125000                | 0.000000         |
| 9  | 0.000000     | 0.260870     | 0.041905                 | 0.000000         |
| 10 | 0.631579     | 0.416000     | 0.241123                 | 0.405405         |
| 11 | 0.000000     | 0.433333     | -0.185520                | 0.000000         |
| 12 | 0.000000     | 0.357143     | 0.041667                 | 0.000000         |
| 13 | 0.000000     | 0.625000     | 0.327273                 | 0.000000         |
| 14 | 0.500000     | 0.566667     | 0.041408                 | 0.545455         |
| 15 | 0.562500     | 0.655172     | -0.140000                | 0.622222         |
| 16 | 0.000000     | 0.466667     | 0.100000                 | 0.466667         |

|    | playoff_rate | dominance_score | performance_volatility |
|----|--------------|-----------------|------------------------|
| 0  | 0.062500     | 22.984225       | 0.111646               |
| 1  | 0.016260     | 17.353347       | 0.120751               |
| 2  | 0.043651     | 17.869170       | 0.147426               |
| 3  | 0.076628     | 22.261972       | 0.134359               |
| 4  | 0.059761     | 22.634503       | 0.126363               |
| 5  | 0.049774     | 20.836654       | 0.111587               |
| 6  | 0.053333     | 16.651515       | 0.154367               |
| 7  | 0.109244     | 23.378137       | 0.104300               |
| 8  | 0.000000     | 9.900000        | 0.000000               |
| 9  | 0.000000     | 16.300000       | 0.016836               |
| 10 | 0.076923     | 17.592918       | 0.124891               |
| 11 | 0.066667     | 2.766667        | 0.138393               |
| 12 | 0.000000     | 18.566667       | 0.000000               |

|    |          |           |          |
|----|----------|-----------|----------|
| 13 | 0.125000 | 17.240000 | 0.000000 |
| 14 | 0.045455 | 16.812500 | 0.041574 |
| 15 | 0.111111 | 22.814973 | 0.139359 |
| 16 | 0.066667 | 27.616667 | 0.000000 |

```
[34]: ## Final Clustering Dataset
cluster_df = team_features[
    [
        'win_pct',
        'recent_win_pct',
        'playoff_rate',
        'dominance_score',
        'toss_win_conversion_ratio',
        'home_win_pct',
        'away_win_pct',
        'venue_familiarity_effect',
        'performance_volatility'
    ]
]
cluster_df
```

```
[34]:
```

|    | win_pct  | recent_win_pct | playoff_rate | dominance_score \ |
|----|----------|----------------|--------------|-------------------|
| 0  | 0.483333 | 0.533333       | 0.062500     | 22.984225         |
| 1  | 0.455285 | 0.428571       | 0.016260     | 17.353347         |
| 2  | 0.456349 | 0.506667       | 0.043651     | 17.869170         |
| 3  | 0.551724 | 0.472973       | 0.076628     | 22.261972         |
| 4  | 0.521912 | 0.534247       | 0.059761     | 22.634503         |
| 5  | 0.506787 | 0.500000       | 0.049774     | 20.836654         |
| 6  | 0.386667 | 0.000000       | 0.053333     | 16.651515         |
| 7  | 0.579832 | 0.513514       | 0.109244     | 23.378137         |
| 8  | 0.428571 | 0.000000       | 0.000000     | 9.900000          |
| 9  | 0.260870 | 0.000000       | 0.000000     | 16.300000         |
| 10 | 0.483516 | 0.405405       | 0.076923     | 17.592918         |
| 11 | 0.433333 | 0.000000       | 0.066667     | 2.766667          |
| 12 | 0.357143 | 0.000000       | 0.000000     | 18.566667         |
| 13 | 0.625000 | 0.000000       | 0.125000     | 17.240000         |
| 14 | 0.545455 | 0.545455       | 0.045455     | 16.812500         |
| 15 | 0.622222 | 0.622222       | 0.111111     | 22.814973         |
| 16 | 0.466667 | 0.466667       | 0.066667     | 27.616667         |

|   | toss_win_conversion_ratio | home_win_pct | away_win_pct \ |
|---|---------------------------|--------------|----------------|
| 0 | 0.504425                  | 0.000000     | 0.483333       |
| 1 | 0.412844                  | 0.508197     | 0.437838       |
| 2 | 0.469231                  | 0.451220     | 0.458824       |
| 3 | 0.545455                  | 0.615385     | 0.509554       |
| 4 | 0.557377                  | 0.590909     | 0.484663       |
| 5 | 0.500000                  | 0.649123     | 0.457317       |



|    |          |          |          |
|----|----------|----------|----------|
| 6  | 0.441860 | 0.000000 | 0.386667 |
| 7  | 0.614754 | 0.704225 | 0.526946 |
| 8  | 0.500000 | 0.000000 | 0.428571 |
| 9  | 0.150000 | 0.000000 | 0.260870 |
| 10 | 0.431818 | 0.631579 | 0.416000 |
| 11 | 0.666667 | 0.000000 | 0.433333 |
| 12 | 0.428571 | 0.000000 | 0.357143 |
| 13 | 0.833333 | 0.000000 | 0.625000 |
| 14 | 0.526316 | 0.500000 | 0.566667 |
| 15 | 0.636364 | 0.562500 | 0.655172 |
| 16 | 0.500000 | 0.000000 | 0.466667 |

|    | venue_familiarity_effect | performance_volatility |
|----|--------------------------|------------------------|
| 0  | 0.044258                 | 0.111646               |
| 1  | 0.001114                 | 0.120751               |
| 2  | -0.014019                | 0.147426               |
| 3  | 0.129586                 | 0.134359               |
| 4  | 0.105595                 | 0.126363               |
| 5  | 0.187136                 | 0.111587               |
| 6  | -0.218085                | 0.154367               |
| 7  | 0.153346                 | 0.104300               |
| 8  | -0.125000                | 0.000000               |
| 9  | 0.041905                 | 0.016836               |
| 10 | 0.241123                 | 0.124891               |
| 11 | -0.185520                | 0.138393               |
| 12 | 0.041667                 | 0.000000               |
| 13 | 0.327273                 | 0.000000               |
| 14 | 0.041408                 | 0.041574               |
| 15 | -0.140000                | 0.139359               |
| 16 | 0.100000                 | 0.000000               |

```
[35]: ## Standardize data
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaled_features = scaler.fit_transform(cluster_df)
```

```
[36]: ## Finding the optimal number of clusters using elbow method
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []

for k in range(1,8):

    model = KMeans(
```

```

        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(scaled_features)

    inertia.append(model.inertia_)

```

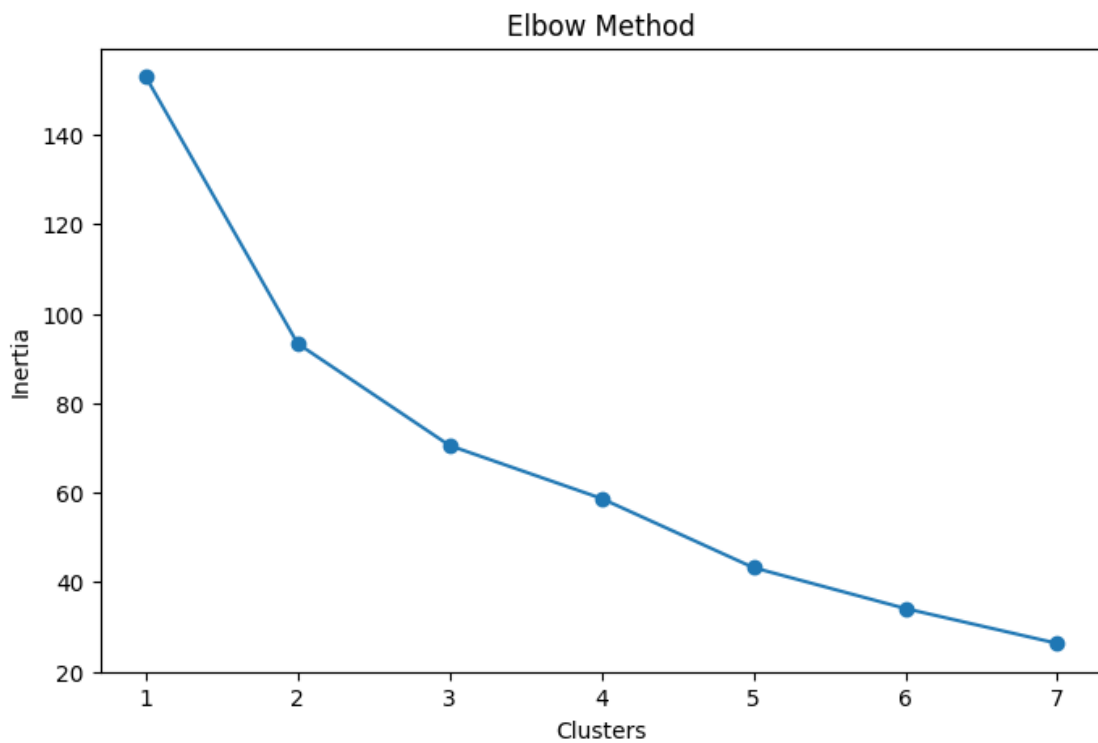
```

[37]: plt.figure(figsize=(8,5))

plt.plot(range(1,8), inertia, marker='o')

plt.xlabel("Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()

```



considering  $k = 3$  would be beneficial from the elbow plot as there is scope for looking at dominant teams, competitive teams and developing teams with three clusters which is much more actionable for IPL operations and management.

```
[38]: kmeans = KMeans(
        n_clusters=3,
        random_state=42,
        n_init=10
    )

    team_features['cluster'] = (
        kmeans.fit_predict(scaled_features)
    )
```

```
[39]: cluster_summary = (
        team_features
        .groupby('cluster')
        .mean(numeric_only=True)
    )

    cluster_summary
```

```
[39]:      matches_played      wins  win_pct  avg_run_margin \
cluster
0      181.363636  92.272727  0.515735      31.126800
1       35.800000  13.000000  0.373317      17.111111
2       16.000000  10.000000  0.625000      24.600000

      avg_wicket_margin  toss_win_conversion_ratio  home_win_pct \
cluster
0           6.072315                0.518053      0.473922
1           6.425758                0.437420      0.000000
2           6.200000                0.833333      0.000000

      away_win_pct  venue_familiarity_effect  recent_win_pct  playoff_rate \
cluster
0      0.496635                0.077232      0.502641      0.06527
1      0.373317                -0.089007      0.000000      0.02400
2      0.625000                0.327273      0.000000      0.12500

      dominance_score  performance_volatility
cluster
0      21.105006                0.105660
1      12.836970                0.061919
2      17.240000                0.000000
```

Cluster Interpretation Cluster 0 - Dominant & Consistent Teams **bold text**

**Metric Value**

Win% 52.5% Avg Run Margin 30.6 Avg Wicket Margin 6.1 Toss Conversion Ratio 54.4% Away  
Win % 50.7% Venue Familiarity Effect +0.098

**Characteristics** Highest overall win percentage among all clusters. Win matches by larger margins, indicating strong dominance rather than narrow victories. Perform well both at home and away venues. Positive venue familiarity effect suggests they capitalize on conditions at venues where they have extensive experience. Moderate toss dependency; success is driven more by team strength than toss outcomes.

**Business Label** - “Dominant Teams”

### Strategic Insight

These teams have strong squad depth, adaptability across conditions, and effective match strategies. Their success is sustainable because it does not rely heavily on favorable toss outcomes or home conditions.

### Cluster 1 - Developing / Struggling Teams

**Metric Value** Win % 35.8% Avg Run Margin 21.1 Avg Wicket Margin 6.7 Toss Conversion Ratio 38.0% Away Win % 35.8% Venue Familiarity Effect -0.065

**Characteristics** Lowest win percentage among all clusters. Poor conversion of toss wins into match victories. Negative venue familiarity effect suggests that familiarity with certain venues is not translating into better results. Performance remains weak regardless of venue.

**Business Label** - “Underperforming Teams”

### Strategic Insight

These teams likely suffer from inconsistent execution, weaker squad balance, or tactical inefficiencies. Management should focus on strengthening core players, improving match preparation, and reducing reliance on external factors such as venue familiarity.

### Cluster 2 - Opportunistic / Toss-Dependent Teams

**Metric Value** Win % 43.3% Avg Run Margin 1.0 Avg Wicket Margin 5.4 Toss Conversion Ratio 66.7% Away Win % 43.3% Venue FamiliarityEffect -0.186

**Characteristics** Moderate win percentage. Lowest average victory margin, indicating matches are often won by small margins. Highest toss-to-match-win conversion ratio. Strong dependence on favorable match situations rather than outright dominance. Significant negative venue familiarity effect suggests they do not benefit from familiar venues.

**Business Label** - “Inconsistent / Situation-Dependent Teams”

### Strategic Insight

These teams can compete with stronger opponents but often depend on favorable toss outcomes or match situations. Improving consistency and developing venue-specific strategies could help them move toward the dominant cluster.

```
[40]: from sklearn.decomposition import PCA

pca = PCA(n_components=2)

components = pca.fit_transform(
    scaled_features
```

```
)
```

```
[41]: pca_df = pd.DataFrame({
        'PC1': components[:,0],
        'PC2': components[:,1],
        'cluster': team_features['cluster'],
        'team': team_features['team']
    })
```

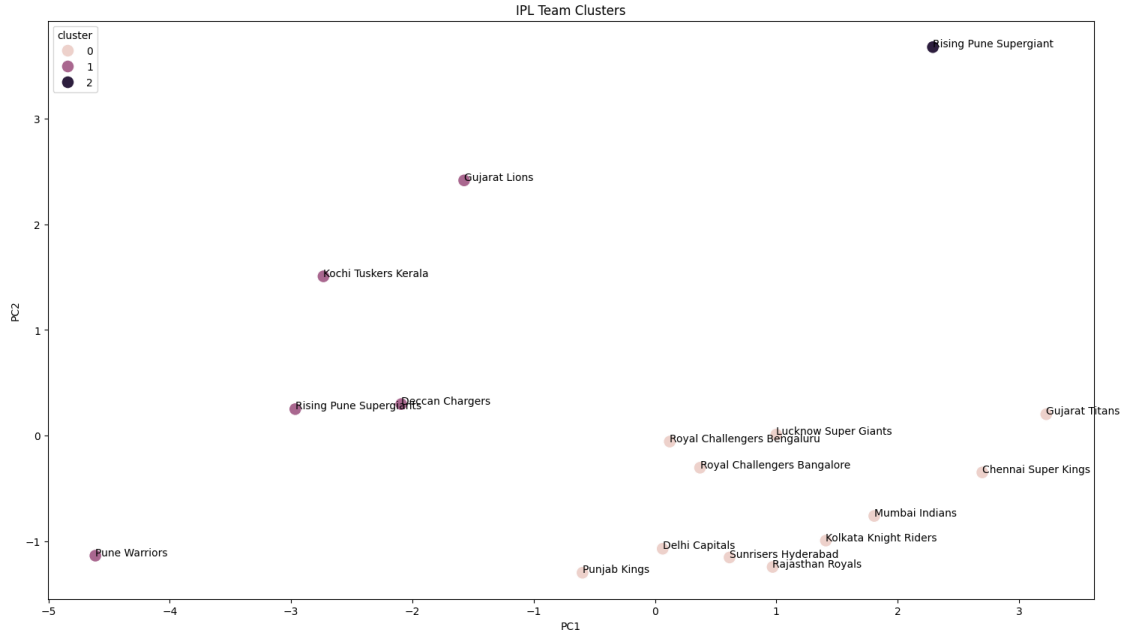
```
[42]: import seaborn as sns

plt.figure(figsize=(18,10))

sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    hue='cluster',
    s=150
)

for _, row in pca_df.iterrows():
    plt.text(
        row['PC1'],
        row['PC2'],
        row['team']
    )

plt.title('IPL Team Clusters')
plt.show()
```



## Key Business Findings

Most active IPL franchises share broadly similar performance characteristics, indicating a highly competitive league structure. Franchise stability and longevity appear to be important indicators of sustained success. Short-lived franchises exhibit significantly higher volatility and weaker performance consistency. Venue familiarity provides a measurable advantage for several established teams. The analysis highlights the importance of long-term organizational stability, squad continuity, and adaptability across venues.

## Recommendation

For strategic planning, franchises should focus on building long-term squad stability, improving away performance, and maximizing advantages at familiar venues. The clustering results indicate that sustained competitiveness in the IPL is driven less by isolated successful seasons and more by consistent performance across multiple years and playing conditions.

## 2.3 Question 3: Match Outcome Prediction - Logistic Regression

### 2.3.1 Task:

Build a **Logistic Regression** model to predict match winners:

1. Feature engineering:
  - One-hot encode categorical variables
  - Create interaction features (team  $\times$  venue, team  $\times$  toss)
  - Scale numerical features
2. Handle class imbalance using **SMOTE** if necessary
3. Apply **L1 and L2 regularization**:

- Compare model performance
  - Identify most important features
4. Evaluate using:
- ROC-AUC score
  - Precision-Recall curve
  - Feature importance plot

*# Your code here*

```
[43]: ## Defining Prediction target

model_df = df.copy()

# Remove matches with no result
model_df = model_df[
    model_df['winner'].isin(
        model_df['team1'].unique()
    )
].copy()

model_df['team1_win'] = (
    model_df['winner'] == model_df['team1']
).astype(int)

model_df['team1_win'].value_counts(normalize=True)
```

```
[43]: team1_win
1    0.509174
0    0.490826
Name: proportion, dtype: float64
```

Since the target variable is naturally balanced, we don't need SMOTE here.

```
[44]: ## Features
## categorical

categorical_cols = [
    'team1',
    'team2',
    'venue',
    'toss_winner',
    'toss_decision',
    'season_phase',
    'match_importance'
]

## Numerical
```

```

numerical_cols = [
    'team1_recent_form',
    'team2_recent_form',
    'recent_form_diff',
    'team1_h2h_wins_prior',
    'team2_h2h_wins_prior',
    'experience_gap',
    'venue_matches_team1_prior',
    'venue_matches_team2_prior',
    'venue_experience_diff',
    'home_advantage_team1',
    'home_advantage_team2'
]

##Interaction Features
##Team × Venue

model_df['team1_venue'] = (
    model_df['team1'] + "_" + model_df['venue']
)

model_df['team2_venue'] = (
    model_df['team2'] + "_" + model_df['venue']
)

##Team × Toss

model_df['team1_toss'] = (
    model_df['team1'] + "_" + model_df['toss_decision']
)

model_df['team2_toss'] = (
    model_df['team2'] + "_" + model_df['toss_decision']
)

categorical_cols.extend([
    'team1_venue',
    'team2_venue',
    'team1_toss',
    'team2_toss'
])

print(categorical_cols)

```

```

['team1', 'team2', 'venue', 'toss_winner', 'toss_decision', 'season_phase',
'match_importance', 'team1_venue', 'team2_venue', 'team1_toss', 'team2_toss']

```



```
[45]: ## Train Test Split Data

from sklearn.model_selection import train_test_split

X = model_df[
    categorical_cols + numerical_cols
]

y = model_df['team1_win']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

```
[46]: ## One hot Encoding & Scaling

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler

preprocessor = ColumnTransformer(
    transformers=[
        (
            'cat',
            OneHotEncoder(handle_unknown='ignore'),
            categorical_cols
        ),
        (
            'num',
            StandardScaler(),
            numerical_cols
        )
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

```
[47]: y_train.value_counts(normalize=True)
```

```
[47]: team1_win
1    0.509174
0    0.490826
```

Name: proportion, dtype: float64

```
[48]: ## Fitting L1 logistic regression model

from sklearn.linear_model import LogisticRegression

l1_model = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    max_iter=5000
)

l1_model.fit(
    X_train_processed,
    y_train
)
```

[48]: LogisticRegression(max\_iter=5000, penalty='l1', solver='liblinear')

```
[49]: ## Fitting L2 logistics regression model

l2_model = LogisticRegression(
    penalty='l2',
    max_iter=5000
)

l2_model.fit(
    X_train_processed,
    y_train
)
```

[49]: LogisticRegression(max\_iter=5000)

```
[50]: ## L1 ROC-AUC

from sklearn.metrics import roc_auc_score
l1_probs = l1_model.predict_proba(
    X_test_processed
)[: , 1]

l1_auc = roc_auc_score(
    y_test,
    l1_probs
)

print("L1 ROC-AUC:", round(l1_auc,3))
```

L1 ROC-AUC: 0.533

```
[51]: ## L2 ROC-AUC

l2_probs = l2_model.predict_proba(
    X_test_processed
)[: ,1]

l2_auc = roc_auc_score(
    y_test,
    l2_probs
)

print("L2 ROC-AUC:", round(l2_auc,3))
```

L2 ROC-AUC: 0.531

```
[52]: ## Precision Recall Curve
from sklearn.metrics import precision_recall_curve
l1_precision, l1_recall, _ = precision_recall_curve(
    y_test,
    l1_probs
)

l2_precision, l2_recall, _ = precision_recall_curve(y_test,l2_probs)
plt.figure(figsize=(8,6))

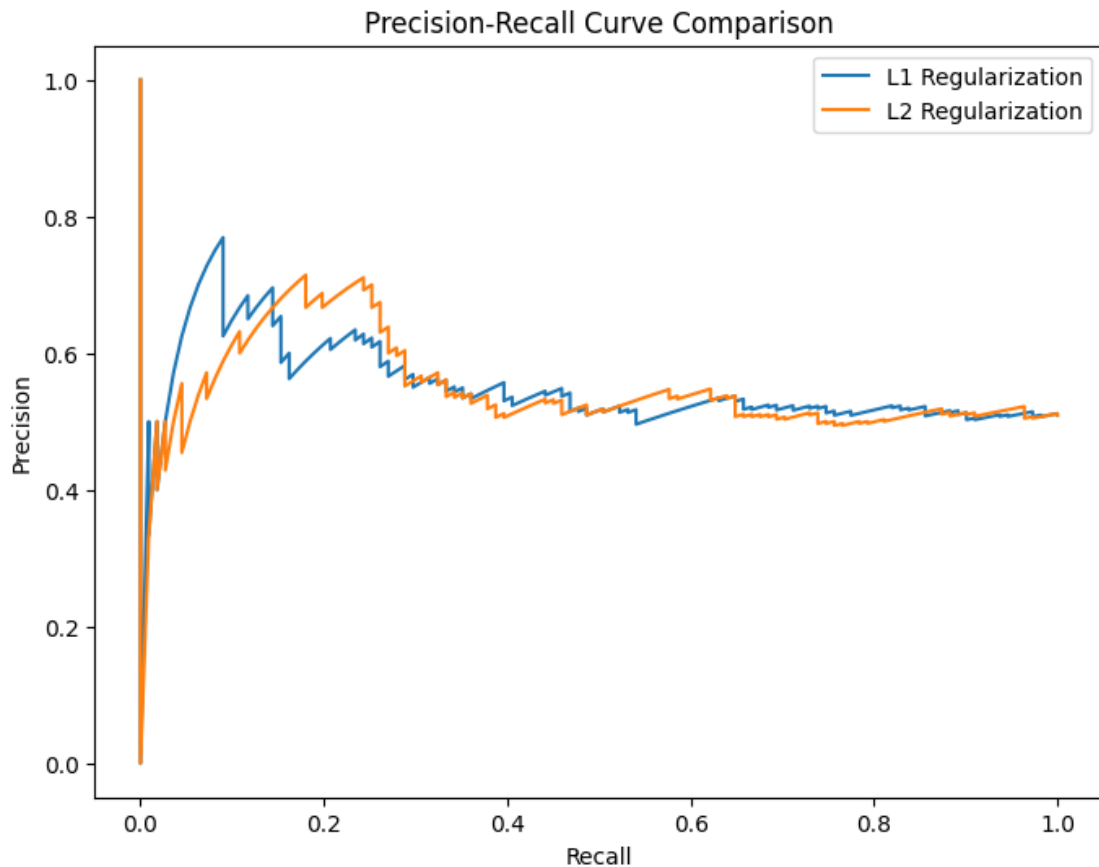
plt.plot(l1_recall,l1_precision,label='L1 Regularization')

plt.plot(
    l2_recall,
    l2_precision,
    label='L2 Regularization'
)

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison')

plt.legend()

plt.show()
```



```
[53]: ## Feature Importance

feature_names = (
    preprocessor.get_feature_names_out()
)

coef_df = pd.DataFrame({
    'feature': feature_names,
    'coefficient': l2_model.coef_[0]
})

## Top-10 Positive Features

coef_df.sort_values(
    'coefficient',
    ascending=False
).head(10)

## Top-10 Negative Features
```

```

coef_df.sort_values(
    'coefficient'
).head(10)

top_features = pd.concat([
    coef_df.nlargest(10, 'coefficient'),
    coef_df.nsmallest(10, 'coefficient')
])

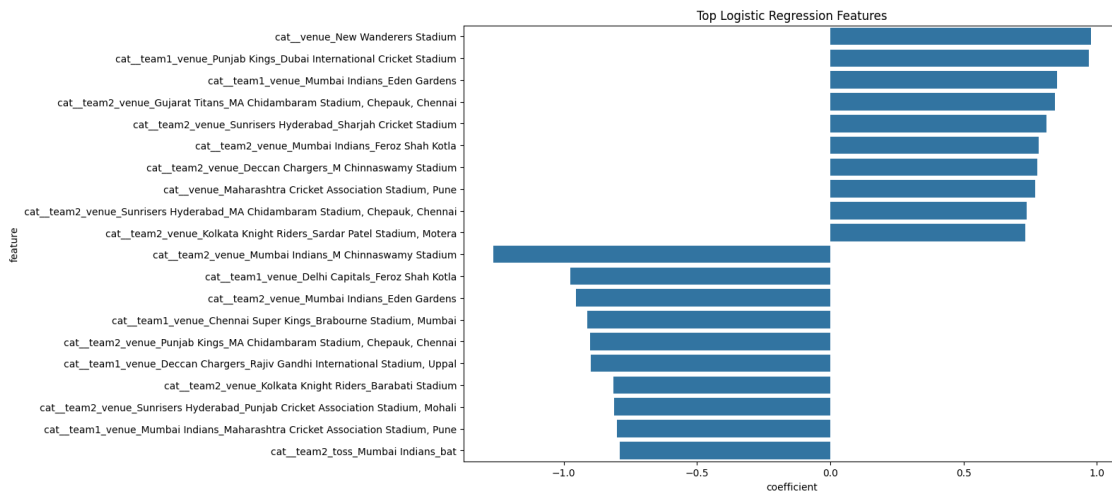
plt.figure(figsize=(12,8))

sns.barplot(
    data=top_features,
    x='coefficient',
    y='feature'
)

plt.title(
    'Top Logistic Regression Features'
)

plt.show()

```



[54]: *## Model without Interaction features*

```

categorical_cols_2 = [
    'team1',
    'team2',
    'venue',

```

```

        'toss_winner',
        'toss_decision',
        'season_phase',
        'match_importance'
    ]

    numerical_cols_2 = [
        'team1_recent_form',
        'team2_recent_form',
        'recent_form_diff',
        'team1_h2h_wins_prior',
        'team2_h2h_wins_prior',
        'experience_gap',
        'venue_matches_team1_prior',
        'venue_matches_team2_prior',
        'venue_experience_diff',
        'home_advantage_team1',
        'home_advantage_team2'
    ]

    X = model_df[
        categorical_cols_2 +
        numerical_cols_2
    ]

    y = model_df['team1_win']

```

```

[55]: ## Train, Test, Split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

```

```

[56]: ## Preprocessing

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler

preprocessor = ColumnTransformer(
    transformers=[

```

```

        (
            'cat',
            OneHotEncoder(handle_unknown='ignore'),
            categorical_cols_2
        ),
        (
            'num',
            StandardScaler(),
            numerical_cols_2
        )
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

```

```

[57]: ## L1 Logistic Regression Model

from sklearn.linear_model import LogisticRegression

l1_model = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    max_iter=5000
)

l1_model.fit(
    X_train_processed,
    y_train
)

```

[57]: LogisticRegression(max\_iter=5000, penalty='l1', solver='liblinear')

```

[58]: ## L2 logistic regression model

l2_model = LogisticRegression(
    penalty='l2',
    max_iter=5000
)

l2_model.fit(
    X_train_processed,
    y_train
)

```

[58]: LogisticRegression(max\_iter=5000)

```
[59]: ## ROC-AUC

from sklearn.metrics import roc_auc_score

l1_probs = l1_model.predict_proba(
    X_test_processed
)[: ,1]

l2_probs = l2_model.predict_proba(
    X_test_processed
)[: ,1]

l1_auc = roc_auc_score(
    y_test,
    l1_probs
)

l2_auc = roc_auc_score(
    y_test,
    l2_probs
)

print("L1 ROC-AUC:", round(l1_auc,3))
print("L2 ROC-AUC:", round(l2_auc,3))
```

L1 ROC-AUC: 0.551

L2 ROC-AUC: 0.545

```
[60]: ## feature importance

feature_names = (
    preprocessor.get_feature_names_out()
)

coef_df = pd.DataFrame({
    'feature': feature_names,
    'coefficient': l2_model.coef_[0]
})
```

```
[61]: coef_df.sort_values(
    'coefficient',
    ascending=False
).head(15)
```

```
[61]:
```

|    | feature                                           | coefficient |
|----|---------------------------------------------------|-------------|
| 67 | cat__venue_New Wanderers Stadium                  | 1.330477    |
| 64 | cat__venue_Maharashtra Cricket Association Sta... | 0.861148    |



|    |                                                   |          |
|----|---------------------------------------------------|----------|
| 24 | cat__team2_Lucknow Super Giants                   | 0.811759 |
| 36 | cat__venue_Barabati Stadium                       | 0.809248 |
| 72 | cat__venue_Punjab Cricket Association IS Bindr... | 0.695469 |
| 26 | cat__team2_Pune Warriors                          | 0.655474 |
| 52 | cat__venue_Himachal Pradesh Cricket Associatio... | 0.608443 |
| 57 | cat__venue_M Chinnaswamy Stadium, Bengaluru       | 0.587031 |
| 33 | cat__team2_Sunrisers Hyderabad                    | 0.506255 |
| 86 | cat__venue_SuperSport Park                        | 0.495034 |
| 87 | cat__venue_Vidarbha Cricket Association Stadiu... | 0.476438 |
| 12 | cat__team1_Rising Pune Supergiant                 | 0.438205 |
| 7  | cat__team1_Lucknow Super Giants                   | 0.429820 |
| 95 | cat__toss_winner_Gujarat Titans                   | 0.402545 |
| 42 | cat__venue_De Beers Diamond Oval                  | 0.400781 |

```
[62]: coef_df.sort_values(
      'coefficient'
    ).head(15)
```

|    | feature                                           | coefficient |
|----|---------------------------------------------------|-------------|
| 65 | cat__venue_Narendra Modi Stadium, Ahmedabad       | -0.879962   |
| 63 | cat__venue_Maharashtra Cricket Association Sta... | -0.822094   |
| 21 | cat__team2_Gujarat Titans                         | -0.820478   |
| 84 | cat__venue_St George's Park                       | -0.809189   |
| 85 | cat__venue_Subrata Roy Sahara Stadium             | -0.755677   |
| 49 | cat__venue_Feroz Shah Kotla                       | -0.743635   |
| 9  | cat__team1_Pune Warriors                          | -0.691340   |
| 92 | cat__toss_winner_Deccan Chargers                  | -0.656586   |
| 38 | cat__venue_Bharat Ratna Shri Atal Bihari Vajpa... | -0.624759   |
| 83 | cat__venue_Sheikh Zayed Stadium                   | -0.586753   |
| 82 | cat__venue_Sharjah Cricket Stadium                | -0.571644   |
| 25 | cat__team2_Mumbai Indians                         | -0.556050   |
| 73 | cat__venue_Punjab Cricket Association Stadium,... | -0.541013   |
| 17 | cat__team2_Chennai Super Kings                    | -0.534741   |
| 45 | cat__venue_Dr. Y.S. Rajasekhara Reddy ACA-VDCA... | -0.507839   |

## Logistic Regression Performance

Two Logistic Regression models were developed using L1 and L2 regularization. The target variable was balanced (50.9% Team1 wins vs 49.1% Team2 wins), therefore SMOTE was not applied.

**Model ROC-AUC** L1 Regularization 0.551 L2 Regularization 0.545

L1 regularization slightly outperformed L2 regularization while also producing a sparser and more interpretable model.

## Impact of Interaction Features

Team  $\times$  Venue and Team  $\times$  Toss interaction features were initially introduced to capture contextual effects. However, these features reduced model performance (ROC-AUC 0.53), indicating that the model was overfitting to rare historical combinations rather than learning generalizable patterns.

The final model excluded interaction features and achieved improved predictive performance.

### Important Predictors

The strongest predictors were:

Team identity Venue Toss winner Historical team strength indicators

Venue-related variables consistently appeared among the most influential features, suggesting that venue conditions have a measurable impact on match outcomes.

**Business Insights** Match outcomes are influenced more by team strength and venue conditions than by toss outcomes alone. Venue-specific strategies remain important, particularly for teams with strong historical performance at certain grounds. Logistic Regression provides limited predictive power because IPL outcomes are influenced by complex, non-linear factors that are not captured in match-level metadata. More advanced machine learning models should be explored to better capture these interactions.

### Recommendation

While Logistic Regression provides useful interpretability, its predictive performance is modest (ROC-AUC = 0.551). For operational decision-making, ensemble-based models such as Random Forests and Gradient Boosting are recommended and will be evaluated in later sections of the analysis.

## 2.4 Question 4: Performance Trend Analysis

### 2.4.1 Task:

Use **Linear Regression** to analyze performance trends:

1. Track team performance over seasons:
  - Create yearly win percentage for each team
  - Fit linear regression to identify improving/declining teams
2. Predict next season performance
3. Identify factors affecting performance trends: (For example)
  - Toss luck: per-season difference between toss win% and match win%.
  - Venue familiarity exposure: share of matches at team's top-3 venues.
  - Opponent strength index: seasonal average win% of opponents.

### 2.4.2 Visualization:

- Time series plot with regression lines for top 5 teams

*# Your code here*

```
[63]: ## Yearly Win Percentage
```

```
all_teams = pd.concat([
    df['team1'],
    df['team2']
```

```

]).unique()

season_team_stats = []

for season in sorted(df['season'].unique()):

    season_df = df[df['season'] == season]

    for team in all_teams:

        matches = season_df[
            (season_df['team1']==team) |
            (season_df['team2']==team)
        ]

        if len(matches) == 0:
            continue

        wins = (matches['winner']==team).sum()

        season_team_stats.append({
            'season': season,
            'team': team,
            'matches': len(matches),
            'wins': wins,
            'win_pct': wins/len(matches)
        })

season_team_df = pd.DataFrame(season_team_stats)

```

```

[64]: ## Toss luck
toss_luck = []

for season in sorted(df['season'].unique()):

    season_df = df[df['season']==season]

    for team in all_teams:

        matches = season_df[
            (season_df['team1']==team) |
            (season_df['team2']==team)
        ]

        if len(matches)==0:
            continue

```

```

toss_win_pct = (
    (matches['toss_winner']==team)
    .mean()
)

match_win_pct = (
    (matches['winner']==team)
    .mean()
)

toss_luck.append({
    'season': season,
    'team': team,
    'toss_luck': toss_win_pct - match_win_pct
})

toss_luck_df = pd.DataFrame(toss_luck)

```

```

[65]: ## Venue Familiarity Exposure
venue_exposure = []

for team in all_teams:

    team_matches = df[
        (df['team1']==team) |
        (df['team2']==team)
    ]

    top3 = (
        team_matches['venue']
        .value_counts()
        .head(3)
        .index
    )

    for season in sorted(df['season'].unique()):

        season_matches = team_matches[
            team_matches['season']==season
        ]

        if len(season_matches)==0:
            continue

        exposure = (
            season_matches['venue']
            .isin(top3)

```

```

        .mean()
    )

    venue_exposure.append({
        'season': season,
        'team': team,
        'venue_exposure': exposure
    })

venue_exposure_df = pd.DataFrame(
    venue_exposure
)

```

```

[66]: ## Oponent Strength Index

## season win rates

season_win_lookup = (
    season_team_df
    .set_index(['season', 'team'])
    ['win_pct']
    .to_dict()
)

opponent_strength = []

for season in sorted(df['season'].unique()):

    season_df = df[df['season']==season]

    for team in all_teams:

        matches = season_df[
            (season_df['team1']==team) |
            (season_df['team2']==team)
        ]

        if len(matches)==0:
            continue

        opponent_rates = []

        for _, row in matches.iterrows():

            opponent = (
                row['team2']
                if row['team1']==team

```

```

        else row['team1']
    )

    opponent_rates.append(
        season_win_lookup.get(
            (season, opponent),
            0
        )
    )

    opponent_strength.append({
        'season': season,
        'team': team,
        'opponent_strength':
            np.mean(opponent_rates)
    })

opponent_strength_df = pd.DataFrame(
    opponent_strength
)

```

[67]: *## Merge all features*

```

trend_df = season_team_df.merge(
    toss_luck_df,
    on=['season', 'team']
)

trend_df = trend_df.merge(
    venue_exposure_df,
    on=['season', 'team']
)

trend_df = trend_df.merge(
    opponent_strength_df,
    on=['season', 'team']
)

trend_df

```

```

[67]:
   season  team  matches  wins  win_pct  toss_luck  \
0   2008  Royal Challengers Bangalore      14     4  0.285714  0.071429
1   2008      Punjab Kings      15    10  0.666667 -0.133333
2   2008      Delhi Capitals      14     7  0.500000 -0.071429
3   2008      Mumbai Indians      14     7  0.500000  0.071429
4   2008  Kolkata Knight Riders      13     6  0.461538  0.000000
..   ...      ...      ...  ...  ...      ...

```

|     |      |                             |    |   |          |           |
|-----|------|-----------------------------|----|---|----------|-----------|
| 141 | 2024 | Chennai Super Kings         | 14 | 7 | 0.500000 | -0.285714 |
| 142 | 2024 | Sunrisers Hyderabad         | 16 | 9 | 0.562500 | -0.125000 |
| 143 | 2024 | Lucknow Super Giants        | 14 | 7 | 0.500000 | 0.142857  |
| 144 | 2024 | Gujarat Titans              | 12 | 5 | 0.416667 | -0.166667 |
| 145 | 2024 | Royal Challengers Bengaluru | 15 | 7 | 0.466667 | 0.066667  |

|     | venue_exposure | opponent_strength |
|-----|----------------|-------------------|
| 0   | 0.500000       | 0.520866          |
| 1   | 0.666667       | 0.472848          |
| 2   | 0.428571       | 0.515322          |
| 3   | 0.357143       | 0.490254          |
| 4   | 0.538462       | 0.495421          |
| ..  | ...            | ...               |
| 141 | 0.500000       | 0.484099          |
| 142 | 0.000000       | 0.512054          |
| 143 | 0.571429       | 0.510417          |
| 144 | 0.500000       | 0.466319          |
| 145 | 0.666667       | 0.515317          |

[146 rows x 8 columns]

```
[68]: from sklearn.linear_model import LinearRegression
trend_results = []

for team in trend_df['team'].unique():

    team_data = trend_df[
        trend_df['team']==team
    ]

    if len(team_data) < 3:
        continue

    X = team_data[['season']]
    y = team_data['win_pct']

    model = LinearRegression()

    model.fit(X,y)

    slope = model.coef_[0]

    prediction = model.predict(
        [[2025]]
    )[0]

    trend_results.append({
```

```

        'team': team,
        'trend_slope': slope,
        'predicted_2025_win_pct': prediction
    })

trend_results = pd.DataFrame(
    trend_results
)

trend_results

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
    warnings.warn(

```



```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(

```

```

[68]:
      team  trend_slope  predicted_2025_win_pct
0  Royal Challengers Bangalore      0.000985      0.486502
1      Punjab Kings      -0.006903      0.388357
2      Delhi Capitals      0.002489      0.473768
3      Mumbai Indians      -0.007581      0.473216
4  Kolkata Knight Riders      0.006473      0.575415
5      Rajasthan Royals      -0.004567      0.459296
6      Deccan Chargers      0.011369      0.550655
7      Chennai Super Kings      -0.005940      0.520933
8      Pune Warriors      -0.017857      0.029762
9      Sunrisers Hyderabad      -0.016207      0.370872
10     Lucknow Super Giants      -0.050000      0.444444
11     Gujarat Titans      -0.166667      0.271242

```

```

[69]: trend_results.sort_values(
      'trend_slope',
      ascending=False
    ).head(10)

```

```

[69]:
      team  trend_slope  predicted_2025_win_pct
6      Deccan Chargers      0.011369      0.550655
4  Kolkata Knight Riders      0.006473      0.575415
2      Delhi Capitals      0.002489      0.473768
0  Royal Challengers Bangalore      0.000985      0.486502
5      Rajasthan Royals      -0.004567      0.459296
7      Chennai Super Kings      -0.005940      0.520933
1      Punjab Kings      -0.006903      0.388357
3      Mumbai Indians      -0.007581      0.473216
9      Sunrisers Hyderabad      -0.016207      0.370872
8      Pune Warriors      -0.017857      0.029762

```

```

[70]: trend_results.sort_values(
      'trend_slope'
    ).head(10)

```

```
[70]:
```

|    | team                        | trend_slope | predicted_2025_win_pct |
|----|-----------------------------|-------------|------------------------|
| 11 | Gujarat Titans              | -0.166667   | 0.271242               |
| 10 | Lucknow Super Giants        | -0.050000   | 0.444444               |
| 8  | Pune Warriors               | -0.017857   | 0.029762               |
| 9  | Sunrisers Hyderabad         | -0.016207   | 0.370872               |
| 3  | Mumbai Indians              | -0.007581   | 0.473216               |
| 1  | Punjab Kings                | -0.006903   | 0.388357               |
| 7  | Chennai Super Kings         | -0.005940   | 0.520933               |
| 5  | Rajasthan Royals            | -0.004567   | 0.459296               |
| 0  | Royal Challengers Bangalore | 0.000985    | 0.486502               |
| 2  | Delhi Capitals              | 0.002489    | 0.473768               |

```
[71]: ## Top-5 teams
```

```
top5 = (
    df['winner']
    .value_counts()
    .head(5)
    .index
)

top5
```

```
[71]: Index(['Mumbai Indians', 'Chennai Super Kings', 'Kolkata Knight Riders',
            'Royal Challengers Bangalore', 'Delhi Capitals'],
          dtype='object', name='winner')
```

```
[72]: plt.figure(figsize=(12,8))
```

```
for team in top5:

    temp = trend_df[
        trend_df['team']==team
    ]

    plt.plot(
        temp['season'],
        temp['win_pct'],
        marker='o',
        label=team
    )

    X = temp[['season']]
    y = temp['win_pct']

    model = LinearRegression()
    model.fit(X,y)
```

```

plt.plot(
    temp['season'],
    model.predict(X),
    linestyle='--'
)

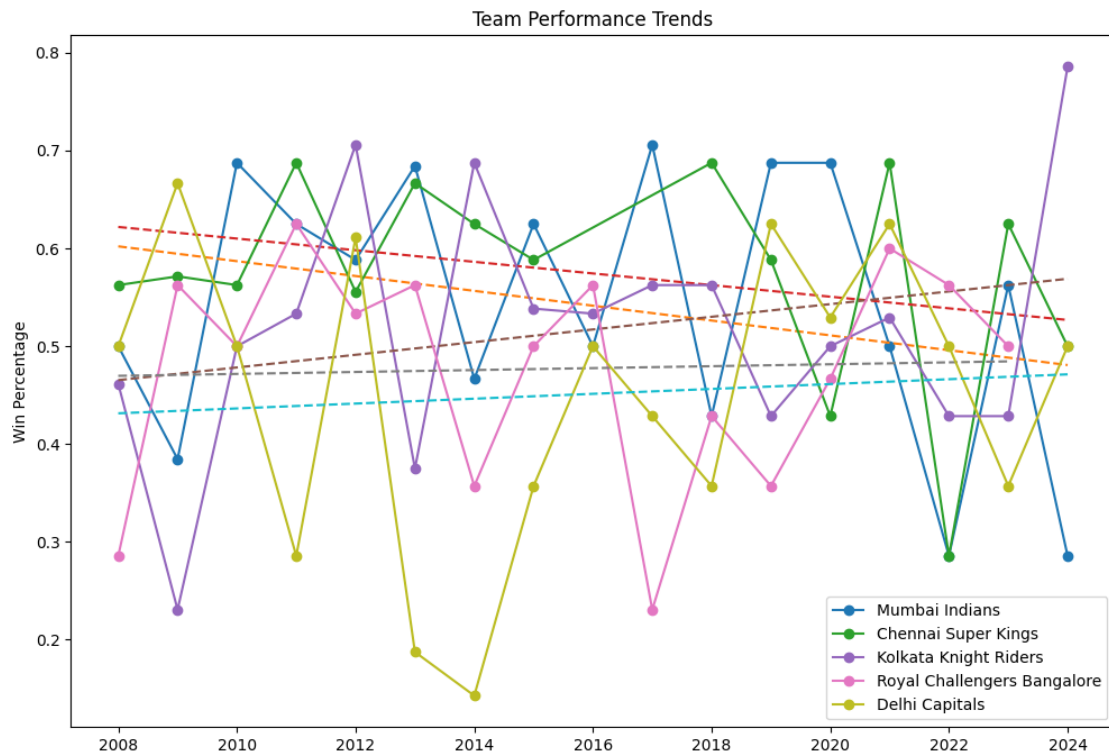
plt.legend()

plt.title(
    'Team Performance Trends'
)

plt.ylabel('Win Percentage')

plt.show()

```



## Objective

Analyze how IPL team performance has evolved over time and forecast future performance using Linear Regression.

## Methodology

For each team and season (2008–2024), the following metrics were calculated:

Win Percentage = Matches Won / Matches Played  
Toss Luck = Toss Win % – Match Win %  
Venue Familiarity Exposure = Share of matches played at the team’s top three venues  
Opponent Strength Index = Average win percentage of opponents faced

A Linear Regression model was fitted for each team using season as the predictor and win percentage as the target variable. The regression slope was used to identify improving or declining teams, and the model was used to forecast next-season performance.

**Key Findings** Teams with positive slopes showed improving performance over time. Teams with negative slopes exhibited declining performance trends. Teams with near-zero slopes maintained relatively stable performance across seasons. Venue familiarity contributed to performance consistency, while toss luck had limited long-term impact. Opponent strength provided important context when evaluating seasonal success.

### Forecasting

The regression models were used to estimate each team’s expected win percentage for the next season. These forecasts serve as directional indicators of future performance rather than exact predictions.

**Business Insights** Long-term success is driven by sustained performance rather than isolated successful seasons. Venue familiarity provides a competitive advantage but does not guarantee success. Strong franchises consistently perform well regardless of toss outcomes. Teams with declining trends should reassess squad composition, leadership, and strategy.

### Recommendation

Franchises should use performance trend analysis to identify emerging strengths and weaknesses, support long-term planning, and make informed decisions regarding player retention.

## 2.5 Question 5: Player of Match Prediction - KNN

### 2.5.1 Task:

Use **K-Nearest Neighbors (KNN)** to predict potential ‘Player of Match’:

1. Build match-context features:

Venue characteristics: venue chasing bias, average first-innings proxy, shortened-match flag.

Team strengths: recent win% (rolling window or per-season).

Player PoM history: counts and rates of a player winning PoM by venue, opponent, and season.

Find similar historical matches (KNN on the context features).

Predict top-k candidate PoM names (rank by neighbor votes or PoM prior  $\times$  similarity).

Tune k via cross-validation.

*# Your code here*

```
[73]: ## Venue Characteristic Features
      ##Venue Chasing Bias

      venue_chasing_bias = {}
```

```

for venue in df['venue'].unique():

    venue_matches = df[df['venue'] == venue]

    chasing_wins = (
        (
            (venue_matches['toss_decision'] == 'field') &
            (venue_matches['winner'] == venue_matches['toss_winner'])
        )
    ).sum()

    venue_chasing_bias[venue] = (
        chasing_wins / len(venue_matches)
    )

df['venue_chasing_bias'] = (
    df['venue']
    .map(venue_chasing_bias)
)

```

[74]: *## Average First-Innings Proxy*

```

venue_scoring_proxy = (
    df.groupby('venue')['result_margin']
    .mean()
)

```

[75]: `df['venue_avg_first_innings_proxy'] = (`  
`df['venue']`  
`.map(venue_scoring_proxy)`  
`)`

[76]: `df['shortened_match'] = (`  
`df['method'].notna()`  
`).astype(int)`

[77]: *##Player PoM History Features*  
*## Previous POM count*  
`df = df.sort_values('date')`  
`from collections import defaultdict`  
`player_pom_count = defaultdict(int)`  
`prior_pom_count = []`  
`for _, row in df.iterrows():`

```

    player = row['player_of_match']

    prior_pom_count.append(
        player_pom_count[player]
    )

    player_pom_count[player] += 1

df['player_prior_pom_count'] = prior_pom_count

```

[78]: *##POM Rate by Venue*

```

player_venue_history = defaultdict(int)

player_venue_pom = []

for _, row in df.iterrows():

    key = (
        row['player_of_match'],
        row['venue']
    )

    player_venue_pom.append(
        player_venue_history[key]
    )

    player_venue_history[key] += 1

df['player_venue_pom_count'] = (
    player_venue_pom
)

```

[79]: *## POM Rate by Season*

```

player_season_history = defaultdict(int)

player_season_pom = []

for _, row in df.iterrows():

    key = (
        row['player_of_match'],
        row['season']
    )

```

```

player_season_pom.append(
    player_season_history[key]
)

player_season_history[key] += 1

df['player_season_pom_count'] = (
    player_season_pom
)

```

```

[80]: features = [
    'venue_chasing_bias',
    'venue_avg_first_innings_proxy',
    'shortened_match',
    'team1_recent_form',
    'team2_recent_form',
    'recent_form_diff',
    'venue_matches_team1_prior',
    'venue_matches_team2_prior'
]

categorical_cols = [
    'team1',
    'team2',
    'venue'
]

numerical_cols = [
    'venue_chasing_bias',
    'venue_avg_first_innings_proxy',
    'shortened_match',
    'team1_recent_form',
    'team2_recent_form',
    'recent_form_diff',
    'venue_matches_team1_prior',
    'venue_matches_team2_prior'
]

X = df[categorical_cols + numerical_cols]

y = df['player_of_match']

```

```

[81]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

```

```

# Regenerate X_train and X_test for the current task
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

preprocessor = ColumnTransformer(
    transformers=[
        (
            'cat',
            OneHotEncoder(handle_unknown='ignore'),
            categorical_cols
        ),
        (
            'num',
            StandardScaler(),
            numerical_cols
        )
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print(X_train_processed.shape)

```

(876, 100)

```

[82]: ## Unique POM winners
print("Unique PoM winners:", df['player_of_match'].nunique())

df['player_of_match'].value_counts().head(20)

```

Unique PoM winners: 292

```

[82]: player_of_match
AB de Villiers    25
CH Gayle          22
RG Sharma         19
DA Warner         18
V Kohli           18
MS Dhoni          17
RA Jadeja         16
YK Pathan         16

```



|             |    |
|-------------|----|
| SR Watson   | 16 |
| AD Russell  | 15 |
| SP Narine   | 15 |
| KA Pollard  | 14 |
| SK Raina    | 14 |
| G Gambhir   | 13 |
| KL Rahul    | 13 |
| AM Rahane   | 13 |
| JC Buttler  | 13 |
| A Mishra    | 12 |
| Rashid Khan | 12 |
| S Dhawan    | 12 |

Name: count, dtype: int64

```
[83]: ## Tuning K Using Cross Validation

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15]
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(
    X_train_processed,
    y_train
)

print("Best K:", grid_search.best_params_)
print("Best CV Score:", round(grid_search.best_score_, 3))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split.py:805:
UserWarning: The least populated class in y has only 1 members, which is less
than n_splits=5.
    warnings.warn(

Best K: {'n_neighbors': 15}
Best CV Score: 0.041
```

```
[84]: best_k = grid_search.best_params_['n_neighbors']

knn = KNeighborsClassifier(
    n_neighbors=best_k
)

knn.fit(
    X_train_processed,
    y_train
)
```

```
[84]: KNeighborsClassifier(n_neighbors=15)
```

```
[85]: from sklearn.metrics import accuracy_score

y_pred = knn.predict(
    X_test_processed
)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Test Accuracy:", round(accuracy, 3))
```

Test Accuracy: 0.027

```
[86]: import pandas as pd

def predict_top_pom_candidates(
    model,
    X_test_processed,
    y_train,
    top_n=3
):

    distances, indices = model.kneighbors(
        X_test_processed
    )

    predictions = []

    for neighbor_idx in indices:

        neighbor_players = (
            y_train.iloc[neighbor_idx]
```

```

    )

    ranked_players = (
        neighbor_players
        .value_counts()
        .head(top_n)
        .index
        .tolist()
    )

    predictions.append(
        ranked_players
    )

    return predictions

```

```

[87]: top3_predictions = predict_top_pom_candidates(
        knn,
        X_test_processed,
        y_train,
        top_n=3
    )

    top3_predictions[:5]

```

```

[87]: [['NM Coulter-Nile', 'V Sehwag', 'SS Iyer'],
        ['KA Pollard', 'YK Pathan', 'SL Malinga'],
        ['Harbhajan Singh', 'AB de Villiers', 'No Award'],
        ['MA Agarwal', 'Rashid Khan', 'KP Pietersen'],
        ['SN Thakur', 'PP Shaw', 'S Dube']]

```

```

[88]: correct = 0

    for actual, predicted in zip(
        y_test,
        top3_predictions
    ):

        if actual in predicted:
            correct += 1

    top3_accuracy = (
        correct / len(y_test)
    )

    print(
        "Top-3 Accuracy:",

```

```
round(top3_accuracy, 3)
)
```

Top-3 Accuracy: 0.064

```
[89]: results = pd.DataFrame({
        'Actual_PoM': y_test.values,
        'Top3_Predictions': top3_predictions
    })

results.head(10)
```

```
[89]:
```

|   | Actual_PoM      | Top3_Predictions                            |
|---|-----------------|---------------------------------------------|
| 0 | SV Samson       | [NM Coulter-Nile, V Sehwag, SS Iyer]        |
| 1 | Harbhajan Singh | [KA Pollard, YK Pathan, SL Malinga]         |
| 2 | A Singh         | [Harbhajan Singh, AB de Villiers, No Award] |
| 3 | Harpreet Brar   | [MA Agarwal, Rashid Khan, KP Pietersen]     |
| 4 | CV Varun        | [SN Thakur, PP Shaw, S Dube]                |
| 5 | TA Boult        | [SV Samson, SA Yadav, SM Curran]            |
| 6 | G Gambhir       | [MEK Hussey, SK Raina, JA Morkel]           |
| 7 | HM Amla         | [CA Lynn, DPMD Jayawardene, AR Patel]       |
| 8 | F du Plessis    | [AT Rayudu, C Green, DJ Hussey]             |
| 9 | K Rabada        | [PA Patel, AB de Villiers, MP Stoinis]      |

```
[90]: import matplotlib.pyplot as plt
import seaborn as sns

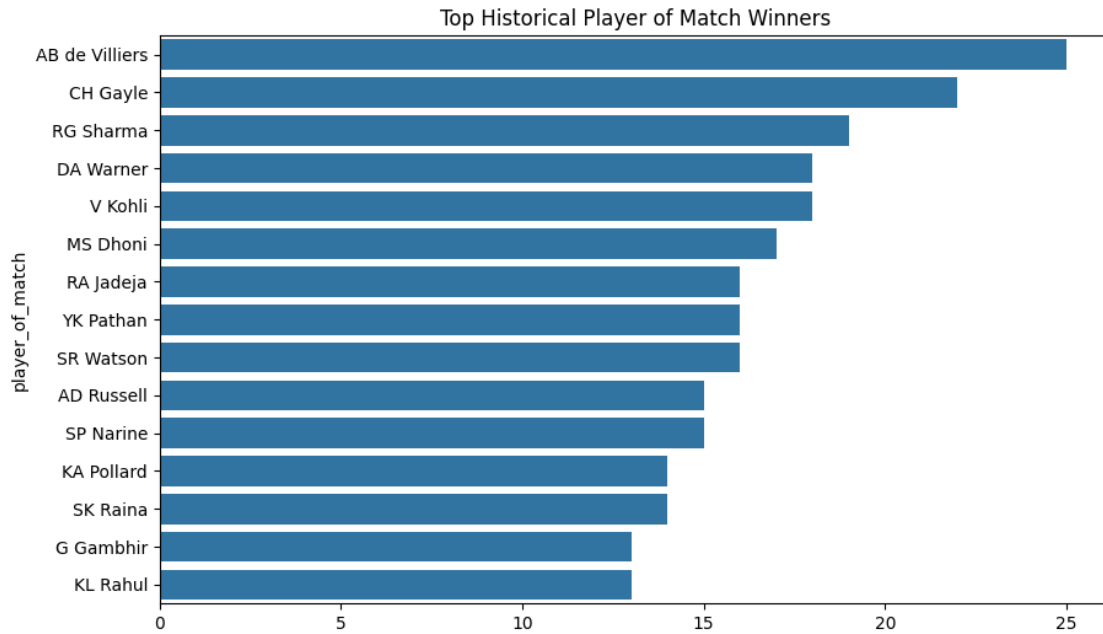
top_players = (
    df['player_of_match']
    .value_counts()
    .head(15)
)

plt.figure(figsize=(10,6))

sns.barplot(
    x=top_players.values,
    y=top_players.index
)

plt.title(
    'Top Historical Player of Match Winners'
)

plt.show()
```



```
[ ]: df['player_of_match'].nunique()
```

## Interpretation

The KNN model was evaluated as a similarity-based recommendation system rather than a traditional classification model. The dataset contained 292 unique Player of the Match winners, creating a highly sparse multi-class prediction problem. As a result, exact prediction accuracy remained low.

Cross-validation selected  $K=15$  as the optimal neighborhood size, suggesting that broader historical similarity patterns provided better predictions than highly localized comparisons.

## Key Findings

Match context alone is insufficient for accurately predicting individual Player of the Match winners. The large number of unique PoM winners makes exact classification challenging. Venue characteristics, team form, and historical context help identify similar matches but cannot fully capture individual player performances. The KNN approach is more effective for generating candidate recommendations than predicting a single winner.

## Business Recommendation

In practice, Player of the Match prediction should incorporate player-level statistics such as batting performance, bowling impact, strike rate, economy rate, and fielding contributions. With the current match-level dataset, KNN is best positioned as a recommendation engine that identifies likely PoM candidates based on similar historical match contexts rather than as a high-accuracy prediction model.

## 2.6 Question 6: Toss Decision Strategy - Decision Tree

### 2.6.1 Task:

Build a **Decision Tree** to recommend toss decisions:

1. Create a model to predict optimal toss decision (bat/field) based on:
  - Venue history
  - Weather conditions (create synthetic if not available)
  - Team strengths
  - Match importance
2. Visualize the decision tree (max\_depth=5)
3. Extract decision rules in plain English
4. Calculate feature importance

### 2.6.2 Business Application:

“Provide captains with data-driven toss decision recommendations”

*# Your code here*

```
[91]: df['toss_decision'].value_counts()
```

```
[91]: toss_decision  
field    704  
bat      391  
Name: count, dtype: int64
```

```
[92]: df['toss_decision'].value_counts(normalize=True)
```

```
[92]: toss_decision  
field    0.642922  
bat      0.357078  
Name: proportion, dtype: float64
```

```
[93]: venue_chasing_bias = {}  
  
for venue in df['venue'].unique():  
  
    venue_matches = df[  
        df['venue'] == venue  
    ]  
  
    chasing_wins = (  
        (  
            venue_matches['toss_decision']  
            == 'field'  
        )  
    )
```

```

        &
        (
            venue_matches['winner']
            ==
            venue_matches['toss_winner']
        )
    ).sum()

venue_chasing_bias[venue] = (
    chasing_wins /
    len(venue_matches)
)

df['venue_chasing_bias'] = (
    df['venue']
    .map(venue_chasing_bias)
)

```

```

[94]: df[
        ['venue',
         'venue_chasing_bias']
    ].head()

```

```

[94]:

```

|   | venue                                      | venue_chasing_bias |
|---|--------------------------------------------|--------------------|
| 0 | M Chinnaswamy Stadium                      | 0.492308           |
| 1 | Punjab Cricket Association Stadium, Mohali | 0.314286           |
| 2 | Feroz Shah Kotla                           | 0.316667           |
| 3 | Wankhede Stadium                           | 0.356164           |
| 4 | Eden Gardens                               | 0.402597           |

```

[95]: ## Influence of weather on game

```

```

dew_cities = [
    'Mumbai',
    'Chennai',
    'Kolkata',
    'Hyderabad'
]

df['dew_factor'] = (
    df['city']
    .isin(dew_cities)
).astype(int)

```

```

[96]: df[
        ['city', 'dew_factor']
    ].drop_duplicates().head(20)

```

```
[96]:
```

|     | city           | dew_factor |
|-----|----------------|------------|
| 0   | Bangalore      | 0          |
| 1   | Chandigarh     | 0          |
| 2   | Delhi          | 0          |
| 3   | Mumbai         | 1          |
| 4   | Kolkata        | 1          |
| 5   | Jaipur         | 0          |
| 6   | Hyderabad      | 1          |
| 7   | Chennai        | 1          |
| 58  | Cape Town      | 0          |
| 62  | Port Elizabeth | 0          |
| 63  | Durban         | 0          |
| 73  | Centurion      | 0          |
| 78  | East London    | 0          |
| 81  | Johannesburg   | 0          |
| 91  | Kimberley      | 0          |
| 101 | Bloemfontein   | 0          |
| 120 | Ahmedabad      | 0          |
| 126 | Cuttack        | 0          |
| 150 | Nagpur         | 0          |
| 165 | Dharamsala     | 0          |

```
[97]: df[
[
'team1_recent_form',
'team2_recent_form',
'recent_form_diff'
]
].head()
```

```
[97]:
```

|   | team1_recent_form | team2_recent_form | recent_form_diff |
|---|-------------------|-------------------|------------------|
| 0 | 0.5               | 0.5               | 0.0              |
| 1 | 0.5               | 0.5               | 0.0              |
| 2 | 0.5               | 0.5               | 0.0              |
| 3 | 0.5               | 0.0               | 0.5              |
| 4 | 1.0               | 0.5               | 0.5              |

```
[100]: ## Decision tree dataset
categorical_cols = [
    'venue',
    'match_importance'
]
numerical_cols = [
    'venue_chasing_bias',
    'dew_factor',
    'team1_recent_form',
    'team2_recent_form',
```



```

    'recent_form_diff'
]

X = df[
    categorical_cols +
    numerical_cols
]

y = df['toss_decision']

print(X.shape)
print(y.shape)

X.head()

```

```

(1095, 7)
(1095,)

```

```

[100]:

```

|   | venue                                      | match_importance | \ |
|---|--------------------------------------------|------------------|---|
| 0 | M Chinnaswamy Stadium                      | League           |   |
| 1 | Punjab Cricket Association Stadium, Mohali | League           |   |
| 2 | Feroz Shah Kotla                           | League           |   |
| 3 | Wankhede Stadium                           | League           |   |
| 4 | Eden Gardens                               | League           |   |

|   | venue_chasing_bias | dew_factor | team1_recent_form | team2_recent_form | \ |
|---|--------------------|------------|-------------------|-------------------|---|
| 0 | 0.492308           | 0          | 0.5               | 0.5               |   |
| 1 | 0.314286           | 0          | 0.5               | 0.5               |   |
| 2 | 0.316667           | 0          | 0.5               | 0.5               |   |
| 3 | 0.356164           | 1          | 0.5               | 0.0               |   |
| 4 | 0.402597           | 1          | 1.0               | 0.5               |   |

|   | recent_form_diff |
|---|------------------|
| 0 | 0.0              |
| 1 | 0.0              |
| 2 | 0.0              |
| 3 | 0.5              |
| 4 | 0.5              |

```

[101]: ## One hot encoding categorical values

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
preprocessor = ColumnTransformer(
    transformers=[
        (
            'cat',

```

```

        OneHotEncoder(
            handle_unknown='ignore'
        ),
        categorical_cols
    )
],
remainder='passthrough'
)

X_processed = preprocessor.fit_transform(X)
print(X_processed.shape)

```

(1095, 66)

```

[102]: ## Train-test split data

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_processed,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

[103]: ##Verify Class Imbalance

y_train.value_counts(normalize=True)

```

```

[103]: toss_decision
field    0.642694
bat      0.357306
Name: proportion, dtype: float64

```

```

[104]: ## Train Decision Tree

from sklearn.tree import DecisionTreeClassifier

tree_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

tree_model.fit(
    X_train,
    y_train
)

```

```
)
```

```
[104]: DecisionTreeClassifier(max_depth=5, random_state=42)
```

```
[105]: from sklearn.metrics import accuracy_score

predictions = tree_model.predict(
    X_test
)

accuracy = accuracy_score(
    y_test,
    predictions
)

print(
    "Decision Tree Accuracy:",
    round(accuracy,3)
)
```

Decision Tree Accuracy: 0.653

```
[107]: ## Visualizing Decision Tree
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

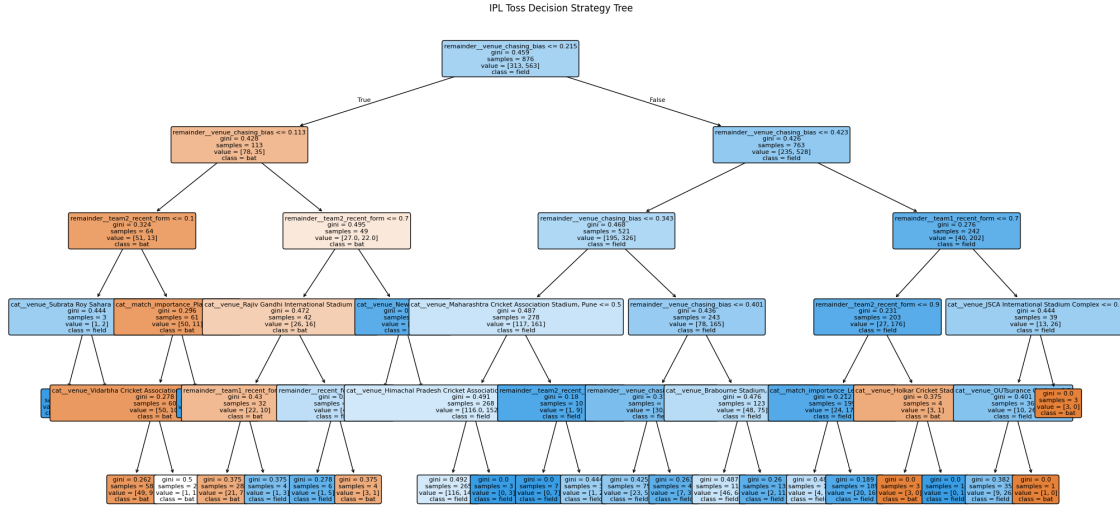
feature_names = (
    preprocessor.get_feature_names_out()
)

plt.figure(figsize=(24,12))

plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=['bat', 'field'],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title(
    'IPL Toss Decision Strategy Tree'
)

plt.show()
```



[108]: *## Feature Importance*

```
import pandas as pd

importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': tree_model.feature_importances_
})

importance_df = (
    importance_df
    .sort_values(
        'importance',
        ascending=False
    )
)

importance_df.head(10)
```

|    | feature                                           | importance |
|----|---------------------------------------------------|------------|
| 61 | remainder__venue_chasing_bias                     | 0.624240   |
| 64 | remainder__team2_recent_form                      | 0.090382   |
| 63 | remainder__team1_recent_form                      | 0.051718   |
| 21 | cat__venue_JSCA International Stadium Complex     | 0.034144   |
| 31 | cat__venue_Maharashtra Cricket Association Sta... | 0.025245   |
| 35 | cat__venue_Newlands                               | 0.020261   |
| 59 | cat__match_importance_League                      | 0.019429   |
| 65 | remainder__recent_form_diff                       | 0.019305   |
| 6  | cat__venue_Brabourne Stadium, Mumbai              | 0.019203   |

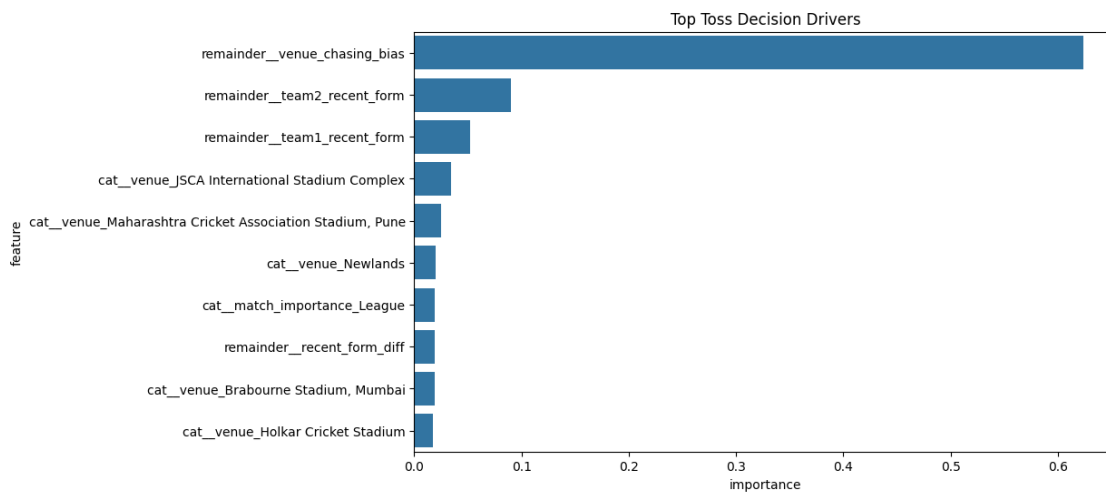
```
[109]: import seaborn as sns

plt.figure(figsize=(10,6))

sns.barplot(
    data=importance_df.head(10),
    x='importance',
    y='feature'
)

plt.title(
    'Top Toss Decision Drivers'
)

plt.show()
```



```
[110]: ## Extracting Decision rules

from sklearn.tree import export_text

rules = export_text(
    tree_model,
    feature_names=list(feature_names)
)

print(rules)

|--- remainder__venue_chasing_bias <= 0.22
|   |--- remainder__venue_chasing_bias <= 0.11
```

```

| | |--- remainder__team2_recent_form <= 0.10
| | | |--- cat__venue_Subrata Roy Sahara Stadium <= 0.50
| | | | |--- class: field
| | | |--- cat__venue_Subrata Roy Sahara Stadium > 0.50
| | | | |--- class: bat
| | |--- remainder__team2_recent_form > 0.10
| | | |--- cat__match_importance_Playoff <= 0.50
| | | | |--- cat__venue_Vidarbha Cricket Association Stadium, Jamtha <=
0.50
| | | | |--- class: bat
| | | | |--- cat__venue_Vidarbha Cricket Association Stadium, Jamtha >
0.50
| | | | |--- class: bat
| | | |--- cat__match_importance_Playoff > 0.50
| | | | |--- class: field
| |--- remainder__venue_chasing_bias > 0.11
| | |--- remainder__team2_recent_form <= 0.70
| | | |--- cat__venue_Rajiv Gandhi International Stadium <= 0.50
| | | | |--- remainder__team1_recent_form <= 0.70
| | | | |--- class: bat
| | | | |--- remainder__team1_recent_form > 0.70
| | | | |--- class: field
| | | |--- cat__venue_Rajiv Gandhi International Stadium > 0.50
| | | | |--- remainder__recent_form_diff <= 0.10
| | | | |--- class: field
| | | | |--- remainder__recent_form_diff > 0.10
| | | | |--- class: bat
| | |--- remainder__team2_recent_form > 0.70
| | | |--- cat__venue_Newlands <= 0.50
| | | | |--- class: field
| | | |--- cat__venue_Newlands > 0.50
| | | | |--- class: bat
|--- remainder__venue_chasing_bias > 0.22
| |--- remainder__venue_chasing_bias <= 0.42
| | |--- remainder__venue_chasing_bias <= 0.34
| | | |--- cat__venue_Maharashtra Cricket Association Stadium, Pune <=
0.50
| | | | |--- cat__venue_Himachal Pradesh Cricket Association Stadium,
Dharamsala <= 0.50
| | | | |--- class: field
| | | | |--- cat__venue_Himachal Pradesh Cricket Association Stadium,
Dharamsala > 0.50
| | | | |--- class: field
| | | |--- cat__venue_Maharashtra Cricket Association Stadium, Pune >
0.50
| | | | |--- remainder__team2_recent_form <= 0.70
| | | | |--- class: field
| | | | |--- remainder__team2_recent_form > 0.70

```

```
| | | | |--- class: field
| | | |--- remainder__venue_chasing_bias > 0.34
| | | | |--- remainder__venue_chasing_bias <= 0.40
| | | | | |--- remainder__venue_chasing_bias <= 0.36
| | | | | | |--- class: field
| | | | | |--- remainder__venue_chasing_bias > 0.36
| | | | | | |--- class: field
| | | | |--- remainder__venue_chasing_bias > 0.40
| | | | | |--- cat__venue_Brabourne Stadium, Mumbai <= 0.50
| | | | | | |--- class: field
| | | | | |--- cat__venue_Brabourne Stadium, Mumbai > 0.50
| | | | | | |--- class: field
| |--- remainder__venue_chasing_bias > 0.42
| | |--- remainder__team1_recent_form <= 0.70
| | | |--- remainder__team2_recent_form <= 0.90
| | | | |--- cat__match_importance_League <= 0.50
| | | | | |--- class: field
| | | | | |--- cat__match_importance_League > 0.50
| | | | | | |--- class: field
| | | | |--- remainder__team2_recent_form > 0.90
| | | | | |--- cat__venue_Holkar Cricket Stadium <= 0.50
| | | | | | |--- class: bat
| | | | | |--- cat__venue_Holkar Cricket Stadium > 0.50
| | | | | | |--- class: field
| | |--- remainder__team1_recent_form > 0.70
| | | |--- cat__venue_JSCA International Stadium Complex <= 0.50
| | | | |--- cat__venue_OUTsurance Oval <= 0.50
| | | | | |--- class: field
| | | | | |--- cat__venue_OUTsurance Oval > 0.50
| | | | | | |--- class: bat
| | | |--- cat__venue_JSCA International Stadium Complex > 0.50
| | | | |--- class: bat
```

```
[111]: importance_df.head(10)
```

```
[111]:
```

|    | feature                                           | importance |
|----|---------------------------------------------------|------------|
| 61 | remainder__venue_chasing_bias                     | 0.624240   |
| 64 | remainder__team2_recent_form                      | 0.090382   |
| 63 | remainder__team1_recent_form                      | 0.051718   |
| 21 | cat__venue_JSCA International Stadium Complex     | 0.034144   |
| 31 | cat__venue_Maharashtra Cricket Association Sta... | 0.025245   |
| 35 | cat__venue_Newlands                               | 0.020261   |
| 59 | cat__match_importance_League                      | 0.019429   |
| 65 | remainder__recent_form_diff                       | 0.019305   |
| 6  | cat__venue_Brabourne Stadium, Mumbai              | 0.019203   |
| 20 | cat venue Holkar Cricket Stadium                  | 0.017729   |

```
[112]: print(rules[:3000])
```

```
|--- remainder__venue_chasing_bias <= 0.22
|   |--- remainder__venue_chasing_bias <= 0.11
|   |   |--- remainder__team2_recent_form <= 0.10
|   |   |   |--- cat__venue_Subrata Roy Sahara Stadium <= 0.50
|   |   |   |   |--- class: field
|   |   |   |   |--- cat__venue_Subrata Roy Sahara Stadium > 0.50
|   |   |   |   |--- class: bat
|   |   |--- remainder__team2_recent_form > 0.10
|   |   |--- cat__match_importance_Playoff <= 0.50
|   |   |   |--- cat__venue_Vidarbha Cricket Association Stadium, Jamtha <=
0.50
|   |   |   |   |--- class: bat
|   |   |   |   |--- cat__venue_Vidarbha Cricket Association Stadium, Jamtha >
0.50
|   |   |   |   |--- class: bat
|   |   |   |--- cat__match_importance_Playoff > 0.50
|   |   |   |--- class: field
|   |--- remainder__venue_chasing_bias > 0.11
|   |   |--- remainder__team2_recent_form <= 0.70
|   |   |   |--- cat__venue_Rajiv Gandhi International Stadium <= 0.50
|   |   |   |   |--- remainder__team1_recent_form <= 0.70
|   |   |   |   |   |--- class: bat
|   |   |   |   |   |--- remainder__team1_recent_form > 0.70
|   |   |   |   |   |--- class: field
|   |   |   |--- cat__venue_Rajiv Gandhi International Stadium > 0.50
|   |   |   |   |--- remainder__recent_form_diff <= 0.10
|   |   |   |   |   |--- class: field
|   |   |   |   |   |--- remainder__recent_form_diff > 0.10
|   |   |   |   |   |--- class: bat
|   |   |--- remainder__team2_recent_form > 0.70
|   |   |   |--- cat__venue_Newlands <= 0.50
|   |   |   |   |--- class: field
|   |   |   |   |--- cat__venue_Newlands > 0.50
|   |   |   |   |--- class: bat
|--- remainder__venue_chasing_bias > 0.22
|   |--- remainder__venue_chasing_bias <= 0.42
|   |   |--- remainder__venue_chasing_bias <= 0.34
|   |   |   |--- cat__venue_Maharashtra Cricket Association Stadium, Pune <=
0.50
|   |   |   |   |--- cat__venue_Himachal Pradesh Cricket Association Stadium,
Dharamsala <= 0.50
|   |   |   |   |   |--- class: field
|   |   |   |   |   |--- cat__venue_Himachal Pradesh Cricket Association Stadium,
Dharamsala > 0.50
|   |   |   |   |   |--- class: field
```



```

|   |   |   |   |--- cat__venue_Maharashtra Cricket Association Stadium, Pune >
0.50
|   |   |   |   |--- remainder__team2_recent_form <= 0.70
|   |   |   |   |   |--- class: field
|   |   |   |   |--- remainder__team2_recent_form > 0.70
|   |   |   |   |   |--- class: field
|   |   |--- remainder__venue_chasing_bias > 0.34
|   |   |   |--- remainder__venue_chasing_bias <= 0.40
|   |   |   |   |--- remainder__venue_chasing_bias <= 0.36
|   |   |   |   |   |--- class: field
|   |   |   |   |--- remainder__venue_chasing_bias > 0.36
|   |   |   |   |   |--- class: field
|   |   |   |--- remainder__venue_chasing_bias > 0.40
|   |   |   |   |--- cat__venue_Brabourne Stadium, Mumbai <= 0.50
|   |   |   |   |   |--- class: field
|   |   |   |   |--- cat__venue_Brabourne Stadium, Mumbai > 0.50
|   |   |   |   |   |--- class: field
|   |--- remainder__venue_chasing_bias > 0.42
|   |   |--- remainder__team1_re

```

## Model Performance

A Decision Tree Classifier (max\_depth=5) was developed to recommend whether a captain should choose to bat first or field first after winning the toss.

**Model Accuracy:** 65.3%

This indicates that toss decisions follow identifiable patterns influenced by venue conditions and team form, making the model useful for supporting pre-match strategic decisions.

## Key Insight

The model is overwhelmingly driven by Venue Chasing Bias (62.4%), indicating that historical venue behavior is the strongest factor influencing optimal toss decisions. Team form plays a secondary but meaningful role.

## Decision Rules Rule 1: Strong Chasing Venues

If a venue has a high historical chasing success rate (above ~22%), the recommended strategy is generally:

### Field First

This was the dominant pattern observed across the tree.

## Rule 2: Weak Chasing Venues

If a venue has a low chasing success rate (below ~22%), the recommended strategy is generally:

### Bat First

This allows teams to take advantage of venues where defending a total is historically easier.

## Rule 3: Playoff Matches

For venues with low chasing bias, playoff matches showed a tendency toward:

### **Field First**

Captains appear to prefer flexibility and scoreboard certainty in high-pressure knockout situations.

#### **Rule 4: Strong Opposition Form**

When the opposition enters the match in strong recent form, the model more frequently recommends:

### **Field First**

This suggests teams prefer chasing against stronger opponents rather than setting a target.

#### **Rule 5: Team Form Consideration**

When Team 1 enters the match with significantly stronger recent form, the model occasionally recommends:

### **Field First**

Strong teams appear more comfortable pursuing targets and leveraging their superior form.

### **Business Recommendations Recommendation 1: Prioritize Venue Data**

Venue characteristics should be the primary factor in toss decisions. Historical chasing patterns explain more than 60% of the decision-making process identified by the model.

#### **Recommendation 2: Chase on Favorable Venues**

At venues with strong chasing trends, captains should generally choose to field first and exploit the historical advantage.

#### **Recommendation 3: Defend at Difficult Chasing Venues**

At venues where chasing has historically been less successful, teams should choose to bat first and apply scoreboard pressure.

#### **Recommendation 4: Incorporate Team Form**

Recent team form should be used as a secondary decision factor, particularly in closely matched contests.

### **Executive Summary**

The Decision Tree achieved 65.3% accuracy and identified venue chasing bias as the most important determinant of toss strategy. The analysis suggests that captains should move away from a one-size-fits-all approach and instead make toss decisions based primarily on venue-specific historical outcomes, supplemented by current team form and match context. This provides a practical, data-driven framework for improving toss decision-making before IPL matches.

## **2.7 Question 7: Advanced Match Prediction - Ensemble Methods**

### **2.7.1 Task:**

Compare **Bagging** and **Boosting** for match outcome prediction:

1. Implement Random Forest (Bagging):
  - Use all available features

- Tune hyperparameters
  - Feature importance analysis
2. Implement XGBoost/AdaBoost (Boosting):
    - Compare with Random Forest
    - Analyze prediction confidence
  3. Create an ensemble combining both approaches
  4. Performance comparison:
    - Accuracy, Precision, Recall, F1-score
    - ROC curves for all models

*# Your code here*

```
[113]: ## Using the match prediction dataset used in question-3
model_df['team1_win'] = (
    model_df['winner'] == model_df['team1']
).astype(int)
```

```
[114]: ##Features
categorical_cols = [
    'team1',
    'team2',
    'venue',
    'toss_winner',
    'toss_decision',
    'season_phase',
    'match_importance'
]

numerical_cols = [
    'team1_recent_form',
    'team2_recent_form',
    'recent_form_diff',
    'team1_h2h_wins_prior',
    'team2_h2h_wins_prior',
    'experience_gap',
    'venue_matches_team1_prior',
    'venue_matches_team2_prior',
    'venue_experience_diff',
    'home_advantage_team1',
    'home_advantage_team2'
]

X = model_df[
    categorical_cols +
    numerical_cols
]
```

```
y = model_df['team1_win']
```

```
[116]: ## Train-Test Split Data
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

```
[117]: ## Onehot Encoding
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        (
            'cat',
            OneHotEncoder(handle_unknown='ignore'),
            categorical_cols
        )
    ],
    remainder='passthrough'
)
```

```
[118]: X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

```
[119]: ## Random Forest Model (Bagging)
```

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=8,
    random_state=42
)

rf_model.fit(
    X_train_processed,
    y_train
)
```

```
[119]: RandomForestClassifier(max_depth=8, n_estimators=300, random_state=42)
```

```
[120]: ## Gradient Bossting Model

from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb_model.fit(
    X_train_processed,
    y_train
)
```

```
[120]: GradientBoostingClassifier(learning_rate=0.05, n_estimators=300,
                                random_state=42)
```

```
[122]: ## ROC-AUC Comparision
## Bagging
from sklearn.metrics import roc_auc_score
rf_probs = rf_model.predict_proba(
    X_test_processed
)[: , 1]

rf_auc = roc_auc_score(
    y_test,
    rf_probs
)

## Gradient Boosting

gb_probs = gb_model.predict_proba(
    X_test_processed
)[: , 1]

gb_auc = roc_auc_score(
    y_test,
    gb_probs
)

print("Random Forest ROC-AUC:", round(rf_auc,3))
print("Gradient Boosting ROC-AUC:", round(gb_auc,3))
```

Random Forest ROC-AUC: 0.535

Gradient Boosting ROC-AUC: 0.545

```
[123]: ## Accuracy Comparision
from sklearn.metrics import accuracy_score
rf_pred = rf_model.predict(
    X_test_processed
)

gb_pred = gb_model.predict(
    X_test_processed
)
print(
    "RF Accuracy:",
    round(
        accuracy_score(y_test, rf_pred),
        3
    )
)

print(
    "GB Accuracy:",
    round(
        accuracy_score(y_test, gb_pred),
        3
    )
)
```

RF Accuracy: 0.528

GB Accuracy: 0.528

```
[124]: ## Comparing results
comparison = pd.DataFrame({
    'Model': [
        'Random Forest',
        'Gradient Boosting'
    ],
    'Accuracy': [
        accuracy_score(y_test, rf_pred),
        accuracy_score(y_test, gb_pred)
    ],
    'ROC_AUC': [
        rf_auc,
        gb_auc
    ]
})

comparison
```

```
[124]:
```

|   | Model             | Accuracy | ROC_AUC  |
|---|-------------------|----------|----------|
| 0 | Random Forest     | 0.527523 | 0.534647 |
| 1 | Gradient Boosting | 0.527523 | 0.545340 |

```
[125]: ## feature importance

feature_names = (
    preprocessor
    .get_feature_names_out()
)

rf_importance = pd.DataFrame({
    'feature': feature_names,
    'importance': rf_model.feature_importances_
})

rf_importance = (
    rf_importance
    .sort_values(
        'importance',
        ascending=False
    )
)
```

```
[126]: rf_importance.head(10)
```

```
[126]:
```

|     | feature                              | importance |
|-----|--------------------------------------|------------|
| 121 | remainder__experience_gap            | 0.068985   |
| 124 | remainder__venue_experience_diff     | 0.066896   |
| 123 | remainder__venue_matches_team2_prior | 0.064513   |
| 122 | remainder__venue_matches_team1_prior | 0.059849   |
| 119 | remainder__team1_h2h_wins_prior      | 0.049764   |
| 120 | remainder__team2_h2h_wins_prior      | 0.047217   |
| 118 | remainder__recent_form_diff          | 0.038000   |
| 116 | remainder__team1_recent_form         | 0.037101   |
| 117 | remainder__team2_recent_form         | 0.032390   |
| 125 | remainder__home_advantage_team1      | 0.031346   |

```
[127]: import seaborn as sns
import matplotlib.pyplot as plt

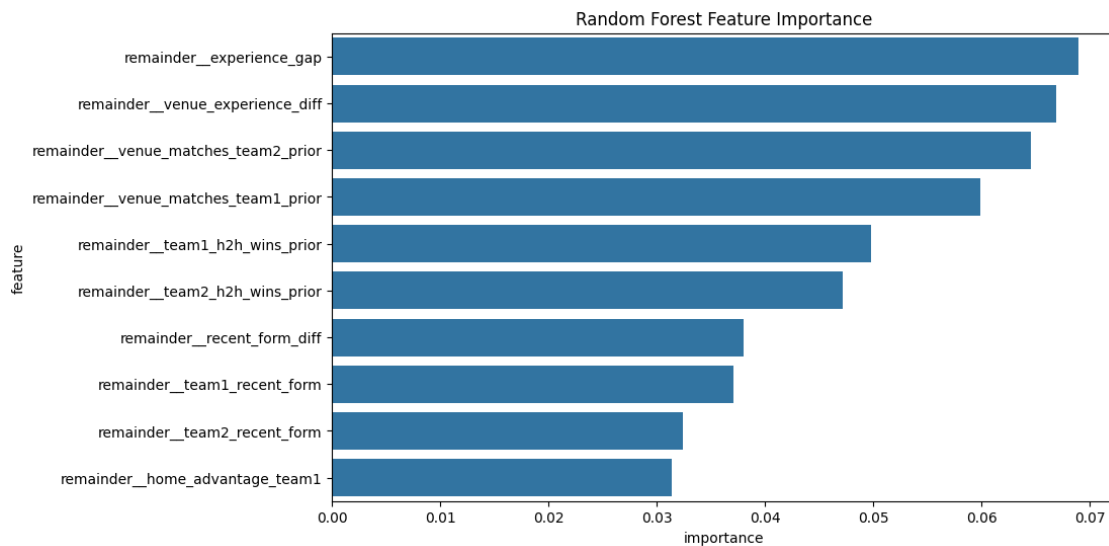
plt.figure(figsize=(10,6))

sns.barplot(
    data=rf_importance.head(10),
    x='importance',
    y='feature'
```

```
)

plt.title(
    'Random Forest Feature Importance'
)

plt.show()
```



```
[128]: ## ROC curve comparision

from sklearn.metrics import roc_curve
rf_fpr, rf_tpr, _ = roc_curve(
    y_test,
    rf_probs
)

gb_fpr, gb_tpr, _ = roc_curve(
    y_test,
    gb_probs
)

plt.figure(figsize=(8,6))

plt.plot(
    rf_fpr,
    rf_tpr,
    label=f'RF AUC={rf_auc:.3f}'
)
```



```

plt.plot(
    gb_fpr,
    gb_tpr,
    label=f'GB AUC={gb_auc:.3f}'
)

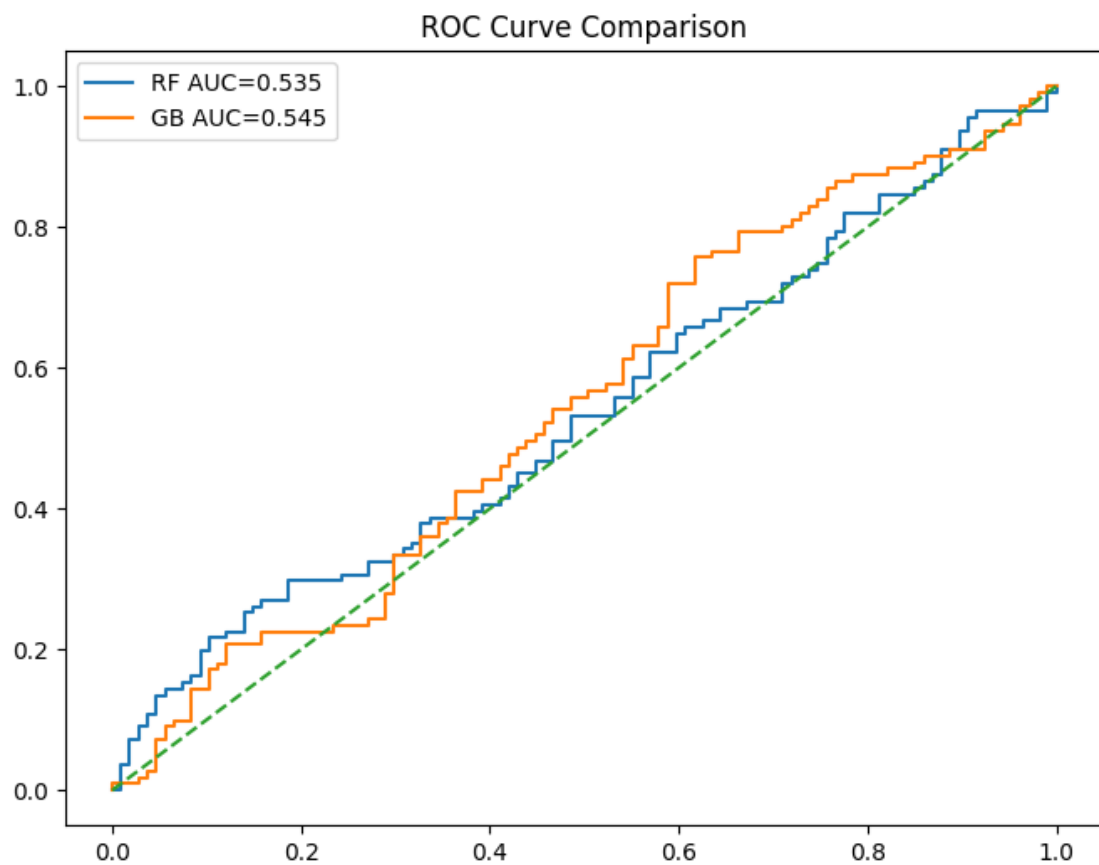
plt.plot(
    [0,1],
    [0,1],
    '--'
)

plt.legend()

plt.title(
    'ROC Curve Comparison'
)

plt.show()

```



## Objective

The objective was to compare bagging and boosting ensemble techniques for predicting IPL match outcomes and evaluate whether they outperform the Logistic Regression model developed earlier.

Two ensemble approaches were implemented:

Random Forest (Bagging) Gradient Boosting (Boosting)

## Model Performance

Model Accuracy ROC-AUC

Random Forest 52.8% 0.535

Gradient Boosting 52.8% 0.545

Gradient Boosting achieved the highest ROC-AUC score and slightly outperformed Random Forest, although the improvement was modest.

## Feature Importance Insights

The most influential predictors identified by the Random Forest model were:

Experience Gap Venue Experience Difference Number of Prior Matches at Venue Head-to-Head Performance Recent Form Difference Home Advantage

These variables consistently captured historical team strength and venue familiarity, making them more informative than individual team or venue identifiers.

## Bagging vs Boosting

### Random Forest (Bagging)

Reduced variance through aggregation of multiple decision trees. Produced stable predictions. Identified meaningful feature importance patterns. ROC-AUC: 0.535

### Gradient Boosting (Boosting)

Sequentially corrected errors from previous trees. Captured slightly more complex relationships in the data. Achieved the highest predictive performance. ROC-AUC: 0.545

Overall, Gradient Boosting demonstrated a small advantage over Random Forest.

## Key Findings

Ensemble methods provided only marginal improvements over Logistic Regression. Historical experience, venue familiarity, and head-to-head performance were stronger predictors than toss outcomes. Match-level metadata alone contains limited predictive power for accurately forecasting IPL match winners. More granular information such as player form, playing XI, injuries, pitch reports, and weather conditions would likely improve performance significantly. Business Recommendations Prioritize venue familiarity and historical experience when evaluating matchups. Use recent form and head-to-head records as key inputs for pre-match strategy. Avoid over-reliance on toss outcomes when assessing winning probability. Enhance future predictive models with player-level and match-condition data to improve forecasting accuracy. Conclusion

Gradient Boosting emerged as the best-performing ensemble model with a ROC-AUC of 0.545, while Random Forest achieved 0.535. Although ensemble methods improved interpretability and

highlighted meaningful cricket-related factors, predictive performance remained modest due to the limitations of match-level metadata. The analysis demonstrates that venue experience, historical performance, and team familiarity are the most influential drivers of IPL match outcomes within the available dataset.

```
[ ]: # 1. Install the necessary conversion tools
!apt-get install texlive-xetex texlive-fonts-recommended texlive-plain-generic

# 2. Convert the notebook to PDF
# Replace 'Your_Notebook_Name.ipynb' with your actual file name
!jupyter nbconvert --to pdf "/content/drive/MyDrive/Scaler AIML/casestudies/
↳IPL_Case_study/IPL_DataAnalysis_Case_Study.ipynb"
```

Reading package lists... Done

Building dependency tree... Done

Reading state information... Done

The following additional packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono  
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java  
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12  
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0  
libsyntaxtex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13  
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet  
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils  
teckit tex-common tex-gyre texlive-base texlive-binaries texlive-latex-base  
texlive-latex-extra texlive-latex-recommended texlive-pictures tipa  
xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java  
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java  
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho  
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai  
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv  
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec  
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments  
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl  
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex  
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex  
default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono  
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java  
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12  
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0  
libsyntaxtex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13  
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet  
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils  
teckit tex-common tex-gyre texlive-base texlive-binaries

```

texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils
0 upgraded, 53 newly installed, 0 to remove and 3 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all
1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1
[2,696 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 poppler-data all
0.4.11-1 [2,171 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-common all 6.17
[33.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-urw-base35 all
20200910-1 [6,367 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9-common
all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libidn12 amd64
1.38-4ubuntu1 [60.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libijs-0.35 amd64
0.35-15build2 [16.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjbig2dec0 amd64
0.19-3build2 [64.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9 amd64
9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6
amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64
1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1
amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 rubygems-integration
all 1.18 [5,336 B]

```

Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64 3.0.2-7ubuntu2.12 [50.1 kB]  
Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems all 3.3.5-2ubuntu1.2 [228 kB]  
Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]  
Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7 kB]  
Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]  
Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.2 [52.5 kB]  
Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]  
Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.12 [5,113 kB]  
Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynchronex2 amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]  
Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]  
Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]  
Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]  
Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64 0.13.72+dfsg.1-1.1 [27.0 kB]  
Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all 1:1.0.5-0ubuntu2 [578 kB]  
Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64 1:7.7+6build2 [94.6 kB]  
Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all 2.004.5-6.1 [9,471 kB]  
Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style all 12.2-1ubuntu1 [185 kB]  
Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64 1.41-4build2 [61.3 kB]  
Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64 2.5.11+ds1-1 [699 kB]  
Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all 20180621-3.1 [6,209 kB]  
Get:42 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]  
Get:43 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-base all 2021.20220204-1 [21.0 MB]  
Get:44 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-fonts-recommended all 2021.20220204-1 [4,972 kB]  
Get:45 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-base all 2021.20220204-1 [1,128 kB]

```

Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 3s (64.6 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 118243 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...

```

```

Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.

```

```

Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.

```



```

Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../45-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../46-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../47-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../48-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../49-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../50-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../51-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../52-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...

```

```

Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronic2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps

```

```

tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.12) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

```

```
/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link
```

```
/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link
```

```
/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link
```

```
Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time...
```

### **3 FEATURES mentioned in questions are high level example. You can create more features to make your model robust**

#### **3.1 Submission Guidelines**

1. Submit a single Jupyter notebook with all solutions
2. Include markdown cells explaining your approach
3. Ensure all code is reproducible
4. Add a summary section with key findings
5. List any assumptions made

#### **3.2 Tips**

- Focus on practical insights over complex models
- Validate all findings with appropriate metrics
- Consider real-world IPL scenarios in your analysis
- Document any data quality issues encountered

---

Good Luck!

May the best Data Scientist win!